



International Islamic University Chittagong

IIUC_StrawHats

Afrin Hasan Safin, Jillur Rahman, Md. Arif

2025-11-21

1 Contest	1
2 Mathematics	1
2.1 Trigonometry	1
2.2 Geometry	1
2.3 Sums	1
2.4 Bitwise Formulas	1
2.5 Trivia	1
3 Data structures	1
4 Number Theory	2
4.1 Sieve and Prime Utilities	2
4.2 Modular Arithmetic	4
4.3 Combinatorics	4
4.4 Big Integer Utilities	4
4.5 Others	5
4.6 More Number Theory	5
5 Graph	5
5.1 DSU and Components	6
5.2 Shortest Paths	6
5.3 Minimum Spanning Tree	7
5.4 Topological Sorting	7
6 Geometry	7
6.1 Geometric Basics	7
6.2 Circles	8
6.3 Misc. Point Set Problems	9
7 Basics and Misc	9
7.1 Basics.	9
7.2 Misc.	10
8 Full template	10
8.1 Arif's template	10

Contest (1)

```
template.cpp
de5d7b, 41 lines

#include<bits/stdc++.h>
using namespace std;

#define fastIO ios_base::sync_with_stdio(0);
cin.tie(0);cout.tie(0)

typedef long long ll;
typedef vector<ll> vl;
typedef pair<ll, ll> pll;
typedef priority_queue<ll, vector<ll>, greater
<ll>> GPQ;

const int inf = 2e9;
const ll INF = 9e18;
```

```
1 #define sz(n) (int)n.size()
1 #define YES cout << "YES\n"
1 #define NO cout << "NO\n"
1
1 #define ff first
1 #define ss second
1 #define nl '\n'
1 #define sp " "
1 #define all(x) (x).begin(), (x).end()
1 #define pb push_back
1 #define forcin(p) for (auto &x : p) cin >> x
1 #define mem(a, b) memset(a, b, sizeof(a))
1
1 void solve()
1 {
1 }
2
2 int main() {
2     fastIO;
2
2     int t=1, tc = 1;
2     // cin >> t;
2     while(t--){
2         // cout << "Case " << tc++ << ": "; //
2         for case
2         solve();
2     }
2 }
```

Mathematics (2)

2.1 Trigonometry

$$\sin(v+w) = \sin v \cos w + \cos v \sin w$$

$$\cos(v+w) = \cos v \cos w - \sin v \sin w$$

$$\tan(v+w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$

$$\sin v + \sin w = 2 \sin \frac{v+w}{2} \cos \frac{v-w}{2}$$

$$\cos v + \cos w = 2 \cos \frac{v+w}{2} \cos \frac{v-w}{2}$$

$$(V+W) \tan\left(\frac{v-w}{2}\right) = (V-W) \tan\left(\frac{v+w}{2}\right)$$

V, W are sides opposite to angles v, w .

$$a \cos x + b \sin x = r \cos(x - \phi)$$

$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}$, $\phi = \text{atan2}(b, a)$.

2.2 Geometry

2.2.1 Triangles

Side lengths: a, b, c

$$\text{Semiperimeter: } p = \frac{a+b+c}{2}$$

$$\text{Area: } A = \sqrt{p(p-a)(p-b)(p-c)}$$

$$\text{Circumradius: } R = \frac{abc}{4A}$$

$$\text{Inradius: } r = \frac{A}{p}$$

Length of median (divides triangle into two equal-area triangles):

$$m_a = \frac{1}{2} \sqrt{2b^2 + 2c^2 - a^2}$$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc \left[1 - \left(\frac{a}{b+c} \right)^2 \right]}$$

Law of sines, cosines & tangents:

$$\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R} \dots (1)$$

$$a^2 = b^2 + c^2 - 2bc \cos \alpha \dots (2)$$

$$\frac{a+b}{a-b} = \frac{\tan((\alpha+\beta)/2)}{\tan((\alpha-\beta)/2)} \dots (3)$$

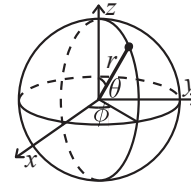
2.2.2 Quadrilaterals

With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2 f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180° , $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$

2.2.3 Spherical coordinates



$$x = r \sin \theta \cos \phi \quad r = \sqrt{x^2 + y^2 + z^2}$$

$$y = r \sin \theta \sin \phi \quad \theta = \arccos(z / \sqrt{x^2 + y^2 + z^2})$$

$$z = r \cos \theta \quad \phi = \text{atan2}(y, x)$$

2.3 Sums

$$c^a + c^{a+1} + \dots + c^b = \frac{c^{b+1} - c^a}{c - 1}, \quad c \neq 1$$

$$1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

$$1^4 + 2^4 + 3^4 + \dots + n^4 = S_2 \times \frac{3n^2 + 3n - 1}{5} = S_4$$

$$b \sum_{k=0}^{n-1} (a + kd)r^k = \frac{ab - (a + nd)br^n}{1 - r} + \frac{dbr(1 - r^n)}{(1 - r)^2}$$

To compute $1^k + \dots + n^k$ in $\mathcal{O}(k \lg k + k \lg MOD)$ compute first $t = k + 2$ sums $y_1, \dots, y_t$, then interpolate. Let $P = \prod_{i=1}^t (n - i)$. Then answer for n is

$$\sum_{i=1}^t \frac{P}{n-i} \cdot \frac{(-1)^{t-i} y_i}{(i-1)!(t-i)!}$$

Also $S_k = \frac{1}{k+1} \sum_{j=0}^k (-1)^j \binom{k+1}{j} B_j n^{k+1-j}$ where B_i are Bernoulli numbers.

2.4 Bitwise Formulas

Some properties of bitwise operations:

$$a|b = a \oplus b + a \& b$$

$$a \oplus (a \& b) = (a|b) \oplus b$$

$$b \oplus (a \& b) = (a|b) \oplus a$$

$$(a \& b) \oplus (a|b) = a \oplus b$$

Addition:

$$a + b = a|b + a \& b$$

$$a + b = a \oplus b + 2(a \& b)$$

Subtraction:

$$a - b = ((a \& b) \oplus ((a|b) \oplus a))$$

$$a - b = ((a|b) \oplus b) - ((a|b) \oplus a)$$

$$a - b = (a \oplus (a \& b)) - ((a|b) \oplus a)$$

$$a - b = (a \oplus (a \& b)) - (b \oplus (a \& b))$$

$$a - b = ((a|b) \oplus b) - (b \oplus (a \& b))$$

Working rules for logarithms:

$$1. \log_b(mn) = \log_b m + \log_b n$$

$$2. \log_b\left(\frac{m}{n}\right) = \log_b m - \log_b n$$

$$3. \log_b(m^r) = r \log_b m$$

$$4. \log_b 1 = 0$$

$$5. \log_b b = 1$$

$$6. \log_b m = \frac{\log_a m}{\log_a b}$$

2.5 Trivia

Pythagorean triples: The Pythagorean triples are uniquely generated by $a = k \cdot (m^2 - n^2)$, $b = k \cdot (2mn)$, $c = k \cdot (m^2 + n^2)$ with $m > n > 0$, $k > 0$, $\gcd(m, n) = 1$, both m, n not odd.

Primes: $p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1000000.

Primitive roots modulo n exists iff $n = 1, 2, 4$ or, $n = p^k, 2p^k$ where p is an odd prime. Furthermore, the number of roots are $\phi(\phi(n))$.

To Find Generator g of M , factor $M - 1$ and get the distinct primes p_i . If $g^{(M-1)/p_i} \not\equiv 1 \pmod{M}$ for each p_i then g is a valid root. Try all g until a hit is found (usually found very quick).

Esitmates: $\sum_{d|n} d = O(n \log \log n)$.

Prime count: 5133 upto 5e4. 9592 upto 1e5. 17984 upto 2e5. 78498 upto 1e6. 5761455 upto 1e8.

max NOD $\leq n$: 100 for $n = 5e4$. 500 for $n = 1e7$. 2000 for $n = 1e10$. 200000 for $n = 1e19$.

max Unique Prime Factors: 6 upto 5e5. 7 upto 9e6. 8 upto 2e8. 9 upto 6e9. 11 upto 7e12. 15 upto 3e19.

Quadratic Residue: $(\frac{a}{p})$ is 0 if $p|a$, 1 if a is a quadratic residue, -1 otherwise. Euler: $(\frac{a}{p}) = a^{(p-1)/2} \pmod{p}$ (prime). Jacobi: if $n = p_1^{e_1} \dots p_k^{e_k}$ then $(\frac{a}{n}) = \prod (\frac{a}{p_i})^{e_i}$.

Chicken McNugget. If a, b coprime, there are $\frac{1}{2}(a-1)(b-1)$ numbers not of form $ax + by$ ($x, y \geq 0$), the largest being $ab - a - b$.

Data structures (3)

```
PBDS.cpp
<bits/stdc++.h>                                01f254, 24 lines

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
template < typename PB >
    using pbds = tree < PB, null_type, less < PB
        >, rb_tree_tag,
        tree_order_statistics_node_update >;
// or

typedef tree<PB,null_type,less<PB>,rb_tree_tag
,tree_order_statistics_node_update> pbds;

example =>
* - less<data_type> --> Increasingly sorted
  set
* - less_equal<data_type> --> Increasingly
  multiset
* - greater<data_type> --> Decreasingly
  sorted set
* - greated_equal<data_type> --> Decreasingly
  sorted multiset

order_of_key(k): Number of items strictly
smaller than k
name.order_of_key(100);
find_by_order(k): returns an iterator , K-th
element in a set ( Counting from zero)
*name.find_by_order(5);
```

```
Path of the File that needs to be renamed in
case of error in including PBDS header
files: C:\Program Files\mingw-w64\x86_64
-8.1.0-posix-seh-rt_v6-rev0\mingw64\lib\
gcc\x86_64-w64-mingw32\8.1.0\include\c
++\ext\pb_ds\detail\resize_policy
// rename a file which has .hpp0000...lots of
numbers to .hpp
```

Number Theory (4)

4.1 Sieve and Prime Utilities

```
ETFsieve.cpp
Description: Computes Euler's Totient Function (phi)
for all numbers up to N using a sieve. phi(n) counts the
integers from 1 to n that are coprime with n.
Usage: etf(); // fills the phi array
int val = phi[97]; // retrieves phi(97)
Memory:  $O(N)$ 
Time:  $O(N \log \log N)$                                 0905fc, 34 lines

/* Upobir
int phi[100000+5];
bool mark[100000+5];

void sieve(){
    for(int i = 1; i<=100000; i++){
        phi[i] = i;

        for(int i = 2; i<=100000; i++){
            if(mark[i])
                continue;
            phi[i] = i-1;    // i is prime => phi(i) = i-1
            for(int j = i+i; j<=100000; j+=i){
                mark[j] = true;

                phi[j] = phi[j] - phi[j]/i;
            }
        }
    }
}

const int N = 1e5 + 9;
int phi[N];
void etf() {
    for (int i = 1; i < N; i++) phi[i] = i;
    for (int i = 2; i < N; i++) {
        if (phi[i] == i) {
            for (int j = i; j < N; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}

ETFsqrt.cpp
Memory:  $O(\sqrt{n})$  for storing prime factors
Time:  $O(\sqrt{n})$                                 b0183a, 24 lines

vector<int> factorize(int n){
    vector<int> primes;
    for(int i = 2; i*i <= n; i++){
        if(n % i == 0){
            primes.push_back(i);
```

```
            while(n % i == 0)
                n /= i;
        }
    }

    if(n > 1)
        primes.push_back(n);
    return primes;
}

int phi(int n){
    vector<int> primes = factorize(n);

    for(int p : primes){
        n = n - n/p;
    }

    return n;
}
```

```
FactoriseSieve.cpp
Description: Generates primes up to M using a modified sieve (odd-only optimization). Also provides a factor() function that returns the prime factorization of n using the precomputed primes.
Usage: sieve(1000000);
vector<int> f = factor(84); // returns {2,2,3,7}
Memory:  $O(M)$ 
Time: Sieve:  $O(n \log \log n)$ , Factorization:  $O(\text{primesuptosqrt}(n))$                                 876ba8, 38 lines

const int M = 1000000; // max : 9e7
vector<int>primes;
bool marked[M];
bool isprime(int n) {
    if (n == 2)
        return true;
    if (n < 2 || n % 2 == 0)
        return false;
    return marked[n] == false;
}

void sieve(1l n) { //  $O(n \log \log n)$ 
    for (1l i = 3; i * i <= n; i += 2) {
        if (marked[i] == false) // i is a prime
        {
            for (1l j = i * i; j <= n; j += i) {
                marked[j] = true;
            }

            for (int i = 0; i < n; i++) {
                if (isprime(i)) {
                    primes.push_back(i);
                }
            }
        }
    }

    vector < int > factor(int n) {
        vector < int > fact;
        for (int i = 0; primes[i] * primes[i] <= n; i++) {
            while (n % primes[i] == 0) {
                n /= primes[i];
                fact.pb(primes[i]); // cnt korbo and comment
            }
        }
        if (n > 1) fact.pb(n);
    }
```

```
        return fact;
        // primes[i] is a , loop cnt is p, a^p
    }
}

FactoriseSqrt.cpp
Description: Returns the prime factorization of a number 'n' as a vector of primes, including multiplicities.
Usage: vector<long long> factors = factorise(60); // returns {2, 2, 3, 5}
Memory:  $O(\sqrt{n})$  for storing factors
Time:  $O(\sqrt{n})$                                 387535, 19 lines

vector<1l> factorise(1l n){
    vector<1l> ret;
    if(n == 1)
        return ret;

    for(1l i = 2; i*i <= n; i++){
        if(n % i == 0){
            while(n % i == 0){
                n /= i;
                ret.push_back(i);
            }
        }
    }

    if(n != 1)
        ret.push_back(n);

    return ret;
}
```

```
NODandSODsieve.cpp
Description: Precomputes Number of Divisors (NOD) and Sum of Divisors (SOD) for all numbers up to nmax using a modified sieve. Then prints tau(n) and sigma(n) for a given n.
Memory:  $O(N)$ 
Time:  $O(N \log N)$                                 4ac22e, 29 lines

const int nmax = 1e5 + 10;

int NOD[nmax];
int SOD[nmax];

void sieve(){ //  $O(N \log N)$ 
    for(int i = 1; i<nmax; i++){
        for(int m = i; m < nmax; m += i){
            NOD[m]++;
            SOD[m] += i;
        }
    }
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);

    sieve();

    1l n = 100;

    cout<<"tau "<<NOD[n]<<endl;
    cout<<"sigma "<<SOD[n]<<endl;

    return 0;
}
```

NodSqrt.cpp
Description: Computes the number of divisors of a number n using its prime factorization.
Usage: ll divisors = NOD(100); returns 9
Memory: $\mathcal{O}(1)$
Time: $\mathcal{O}(\sqrt{n})$

```
11 NOD(ll n) {
    ll nod = 1;
    for (ll i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            ll p = 0;
            while (n % i == 0) {
                n /= i;
                p++;
            }
            nod *= (p + 1);
        }
    }
    if (n != 1)
        nod *= 2;
    return nod;
}
```

PrimeSieve.cpp
Description: Generates all prime numbers up to n using a simple sieve. Marks non-primes in a boolean array and stores primes in a vector.
Usage: sieve(1000000);
for (int p : primes) cout << p << ' ' ;
Memory: $\mathcal{O}(M)$
Time: $\mathcal{O}(n \log \log n)$

```
const int M = 1000000; // max : 9e7
vector<int>primes;
bool marked[M];
bool isprime(int n) {
    if (n == 2)
        return true;
    if (n < 2 || n % 2 == 0)
        return false;
    return marked[n] == false;
}
void sieve(ll n) { // O(n log log n)
    for (ll i = 3; i * i <= n; i += 2) {
        if (marked[i] == false) // i is a prime
        {
            for (ll j = i * i; j <= n; j += i) {
                marked[j] = true;
            }
        }
        for (int i = 0; i < n; i++) {
            if (isprime(i)) {
                primes.push_back(i);
            }
        }
    }
}
```

SPFandHPF.cpp
Description: Computes smallest prime factor (spf) for all numbers up to MAXN using a sieve. Allows quick factorization of any number <= MAXN.
Usage: sieve(); // precompute spf
auto factors = getFactorization(100); // returns prime factors of 100
Memory: $\mathcal{O}(MAXN)$
Time: $\mathcal{O}(MAXN \log \log MAXN)$ for sieve, $\mathcal{O}(\log n)$ per factorization

```
int const MAXN = (int)1e7 + 5;
vector<int> spf(MAXN + 1, 1);
void sieve()
{
    spf[0] = 0;
    for (int i = 2; i <= MAXN; i++) {
        if (spf[i] == 1) {
            for (int j = i; j <= MAXN; j += i)
            {
                if (spf[j] == 1)
                    spf[j] = i;
            }
        }
    }
}
vector<int> getFactorization(int x) // with spf
{
    vector<int> ret;
    while (x != 1) {
        ret.push_back(spf[x]);
        x = x / spf[x];
    }
    return ret;
}
```

SegmentedSieve.cpp
Description: Generates all primes in (l, r) using a segmented sieve.
Usage: auto primes1 = sieve(100); primes up to 100
auto primes2 = segmentedSieve(1000, 2000); primes in (1000 to 2000)
Memory: $\mathcal{O}(r - l + \sqrt{r})$
Time: sieve $\mathcal{O}(\text{limit} \log \log \text{limit})$, segmentedSieve $\mathcal{O}((r - l) \log r + \sqrt{r})$

```
// Generate all primes up to limit using sieve of eratosthenes
vector<int> sieve(int limit) {
    vector<bool> is_prime(limit + 1, true);
    is_prime[0] = is_prime[1] = false;
    for (int p = 2; p * p <= limit; ++p) {
        if (is_prime[p]) {
            for (int i = p * p; i <= limit; i += p) {
                is_prime[i] = false;
            }
        }
    }
    vector<int> primes;
    for (int p = 2; p <= limit; ++p) {
        if (is_prime[p]) {
            primes.push_back(p);
        }
    }
    return primes;
}
```

```
// Generate all primes from l to r using segmented sieve in O((r - l) log (r) + sqrt(r))
vector<ll> segmented_sieve(ll l, ll r) {
    if (l == 1) {
        l++;
    }
    int limit = sqrtl(r);
    while ((ll) limit * limit <= r) limit++;
```

```
while ((ll) limit * limit > r) limit--;
auto primes = sieve(limit);
vector<bool> is_prime(r - l + 1, true);
for (ll p : primes) {
    ll start = max((ll)p * p, (ll)(l + p - 1) / p * p);
    for (ll j = start; j <= r; j += p) {
        is_prime[j - l] = false;
    }
}
vector<ll> vec;
for (ll i = l; i <= r; ++i) {
    if (is_prime[i - l]) {
        vec.push_back(i);
    }
}
return vec;
}
int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    int t;
    cin >> t;
    while (t--) {
        ll l, r;
        cin >> l >> r;
        auto primes = segmented_sieve(l, r);
        for (auto p: primes) {
            cout << p << '\n';
        }
    }
    return 0;
}
```

SodSqrt.cpp
Description: Computes the sum of divisors of a number n using its prime factorization. If $n = p_1^{a_1} * p_2^{a_2} * \dots * p_k^{a_k}$, then $SOD(n) = ((p_1^{a_1+1} - 1) / (p_1 - 1)) * \dots * ((p_k^{a_k+1} - 1) / (p_k - 1))$
Usage: ll sum = SOD(100); returns sum of divisors of 100
Memory: $\mathcal{O}(1)$
Time: $\mathcal{O}(\sqrt{n})$

```
11 SOD(ll n) {
    ll sod = 1;
    for (ll i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            ll cur = i;
            while (n % i == 0) {
                cur *= i, n /= i;
            }
            cur = (cur - 1) / (i - 1);
            sod *= cur;
        }
    }
    if (n != 1) sod *= (n + 1);
    return sod;
}
```

bitsetSieve.cpp
Description: Provides a memory-efficient implementation of the Sieve of Eratosthenes using bitwise operations. Includes 'sieve(n)' to generate all primes up to 'n' and 'isPrime(num)' to check primality of a number in $\mathcal{O}(1)$ time.
Memory: $\mathcal{O}(n/64)$

Time: $\mathcal{O}(n \log \log n)$ for sieve, $\mathcal{O}(1)$ for isPrime

```
#define M 100000000
int marked[M / 64 + 2];

#define on(x) (marked[x / 64] & (1 << ((x % 64) / 2)))
#define mark(x) marked[x / 64] |= (1 << ((x % 64) / 2))

void sieve(int n)
{
    for (int i = 3; i * i < n; i += 2)
    {
        if (!on(i))
        {
            for (int j = i * i; j <= n; j += i)
            {
                mark(j);
            }
        }
    }

    bool isPrime(int num)
    {
        return num > 1 && (num == 2 || ((num & 1) && !on(num)));
    }

    // memory efficient er jonno ei sieve
```

progkriyaSieve.cpp
Description: Marks composite numbers up to n. Only odd numbers are processed to improve speed. 'marked[x] = true' means composite. 'isPrime(x)' checks primality in $\mathcal{O}(1)$. Complexity: Time $\mathcal{O}(n \log \log n)$, Space $\mathcal{O}(n)$.

// from book

sieve upto 1e9.cpp

Memory: $\mathcal{O}(N^{1/2} + \text{blocksize} + \text{primecount})$

Time: Approximately 0.5s for N = 1e9

fe7395, 93 lines

```
// credit: min_25
// takes 0.5s for n = 1e9
vector<int> sieve(const int N, const int Q = 17, const int L = 1 << 15) {
    static const int rs[] = {1, 7, 11, 13, 17, 19, 23, 29};
    struct P {
        P(int p) : p(p) {}
        int p; int pos[8];
    };
    auto approx_prime_count = [] (const int N) -> int {
        return N > 60184 ? N / (log(N) - 1.1) : max(1., N / (log(N) - 1.11)) + 1;
    };

    const int v = sqrt(N), vv = sqrt(v);
    vector<bool> isp(v + 1, true);
    for (int i = 2; i <= vv; ++i) if (isp[i]) {
        for (int j = i * i; j <= v; j += i) isp[j] = false;
    }

    const int rsize = approx_prime_count(N + 30);
    vector<int> primes = {2, 3, 5}; int psize = 3;
    primes.resize(rsize);

    vector<P> sprimes; size_t pbeg = 0;
    int prod = 1;
    for (int p = 7; p <= v; ++p) {
        if (!isp[p]) continue;
        if (p <= Q) prod *= p, ++pbeg, primes[psize++] = p;
        auto pp = P(p);
        for (int t = 0; t < 8; ++t) {
            int j = (p <= Q) ? p : p * p;
            while (j % 30 != rs[t]) j += p << 1;
            pp.pos[t] = j / 30;
        }
        sprimes.push_back(pp);
    }

    vector<unsigned char> pre(prod, 0xFF);
    for (size_t pi = 0; pi < pbeg; ++pi) {
        auto pp = sprimes[pi]; const int p = pp.p;
        for (int t = 0; t < 8; ++t) {
            const unsigned char m = ~(1 << t);
            for (int i = pp.pos[t]; i < prod; i += p) pre[i] &= m;
        }
    }

    const int block_size = (L + prod - 1) / prod * prod;
    vector<unsigned char> block(block_size);
    unsigned char* pblock = block.data();
    const int M = (N + 29) / 30;

    for (int beg = 0; beg < M; beg += block_size, pblock += block_size) {
        int end = min(M, beg + block_size);
        for (int i = beg; i < end; i += prod) {
```

```
        copy(pre.begin(), pre.end(), pblock + i);
    }
    if (beg == 0) pblock[0] &= 0xFE;
    for (size_t pi = pbeg; pi < sprimes.size(); ++pi) {
        auto& pp = sprimes[pi];
        const int p = pp.p;
        for (int t = 0; t < 8; ++t) {
            int i = pp.pos[t]; const unsigned char m = ~(1 << t);
            for (; i < end; i += p) pblock[i] &= m;
        }
        pp.pos[t] = i;
    }
    for (int i = beg; i < end; ++i) {
        for (int m = pblock[i]; m > 0; m &= m - 1) {
            primes[psize++] = i * 30 + rs[__builtin_ctz(m)];
        }
    }
    assert(psize <= rsize);
    while (psize > 0 && primes[psize - 1] > N) --psize;
    primes.resize(psize);
    return primes;
}

int32_t main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    int n, a, b; cin >> n >> a >> b; // n porjontw prime generate kore dibe
    auto primes = sieve(n);
    vector<int> ans;
    for (int i = b; i < primes.size() && primes[i] <= n; i += a) ans.push_back(primes[i]);
    // b theke shuru kore a ghor por por n porjontw prime gulo print korbe
    // input n=10 a=1 b=0 dile buja jabe
    cout << primes.size() << ' ' << ans.size() << '\n';
    for (auto x: ans) cout << x << ' '; cout << '\n';
    return 0;
}

// source link: https://github.com/ShahjalalShohag/code-library/blob/main/Number%20Theory/Sieve%20upto%201e9.cpp
// https://judge.yosupo.jp/problem/enumerate_primes
```

4.2 Modular Arithmetic

mod bin exp.cpp

Description: this mod is bit slow, use *ans*— = *ans/mod * mod*

Memory: $\mathcal{O}(1)$

Time: $\mathcal{O}(\log n)$ for bin exp and modInv, $\mathcal{O}(1)$ for other operations

39f9b5, 19 lines

```
ll bin_exp(ll a, ll n) //O(log(n))
{
    // a ^ n
    ll r = 1LL;
```

```
int m = 1000000007;
// Delete %m if we don't need it
while (n) {
    if (n & 1) r = (r * a) % m, n--;
    else a = (a * a) % m, n /= 2LL;
}
return r;
}

const int mod2 = 1e9 + 7; // faster if const
inline ll MOD(ll a){ return (a % mod2 + mod2) % mod2; }
inline ll modAdd(ll a, ll b) { return MOD(MOD(a) + MOD(b)); }
inline ll modSub(ll a, ll b) { return MOD(MOD(a) - MOD(b)); }
inline ll modMul(ll a, ll b) { return MOD(MOD(a) * MOD(b)); }
inline ll modInv(ll a) { return bin_exp(a, mod2 - 2); }

modLog.cpp
Description: Returns the smallest  $x > 0$  such that  $a^x \equiv b \pmod{m}$  or -1 if no such  $x$  exists. This function can also be used to compute the order of a modulo  $m$  by calling modLog(a,1,m).
Usage: ll x = modLog(2, 8, 13); finds  $x$  such that  $2^x \equiv 8 \pmod{13}$ 
Memory:  $\mathcal{O}(\sqrt{m})$  for the map
Time:  $\mathcal{O}(\sqrt{m})$ 
f73175, 15 lines

#define rep(i,x,n) for(int i=x; i<n; i++)

ll modLog(ll a, ll b, ll m)
{
    ll n = (ll)sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> A;
    while (j <= n && (e = f = e * a % m) != b % m)
        A[e * b % m] = j++;
    if (e == b % m) return j;
    if (__gcd(m, e) == __gcd(m, b))
        rep(i, 2, n + 2) if (A.count(e = e * f % m))
            return n * i - A[e];
    return -1;
}
```

Mint.cpp

Description: Operators for modular arithmetic. You need to set mod to some number first and then you can use the structure.

e4da80, 33 lines

```
const ll mod = 1e9+7; // change to something else
struct Mint {
    ll x;
    Mint() : x(0){}
    Mint(ll x_) : x(x_ % mod){if (x < 0) x += mod;}
    Mint operator+(Mint b) { return Mint((x + b.x) % mod); }
    Mint operator-(Mint b) { return Mint((x - b.x + mod) % mod); }
    Mint operator*(Mint b) { return Mint((x * b.x) % mod); }
```

```
Mint operator/(Mint b) { return *this * invert(b); }
Mint invert(Mint a) {
    ll x, y, g = euclid(a.x, mod, x, y);
    assert(g == 1); return Mint((x + mod) % mod);
}
Mint power(ll e) {
    if (!e) return Mint(1);
    Mint r = *this->power(e / 2); r = r * r;
    return e&1 ? *this * r : r;
};

vector<Mint> fact(1, 1);
vector<Mint> inv_fact(1, 1);

Mint C(int n, int k) {
    if (k < 0 || k > n) return 0;

    while ((int) fact.size() < n + 1) {
        fact.push_back(fact.back() * (int) fact.size());
        inv_fact.push_back( Mint(1) / fact.back()) ;
    }
    return fact[n] * inv_fact[k] * inv_fact[n - k];
}

4.3 Combinatorics
nCr nPr.cpp
Description: Precomputes factorials and inverse factorials modulo MOD up to MAXN. Provides fast computation of nCr and nPr in  $\mathcal{O}(1)$  time after  $\mathcal{O}(\text{MAXN})$  preprocessing.
Usage: precompute();
long long c = nCr(5, 2); // 10
long long p = nPr(5, 2); // 20
Memory:  $\mathcal{O}(\text{MAXN})$ 
Time: Precompute  $\mathcal{O}(\text{MAXN})$ , Queries  $\mathcal{O}(1)$ 
3c5c7a, 40 lines

const int MAXN = 2000005; // change as needed
const long long MOD = 1000000007;

vector<long long> fact(MAXN), inv_fact(MAXN);

long long modpow(long long a, long long e) {
    long long r = 1;
    while (e) {
        if (e & 1) r = r * a % MOD;
        a = a * a % MOD;
        e >>= 1;
    }
    return r;
}

void precompute() {
    fact[0] = 1;
    for (int i = 1; i < MAXN; ++i) {
        fact[i] = fact[i - 1] * i % MOD;
    }

    long long cur = fact[MAXN - 1];
    cur = modpow(cur, MOD - 2); // modular inverse of fact[MAXN-1]
    inv_fact[MAXN - 1] = cur;
```



```
for (int i = MAXN - 2; i >= 0; --i) {
    inv_fact[i] = inv_fact[i + 1] * (i + 1) % MOD;
}

long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * inv_fact[r] % MOD * inv_fact[n - r] % MOD;
}

long long nPr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * inv_fact[n - r] % MOD;
}
```

4.4 Big Integer Utilities

stringAdd.cpp
Description: Adds two large integers represented as strings. Assumes both strings are of the same length. Returns the sum as a string.
Memory: $\mathcal{O}(n)$
Time: $\mathcal{O}(n)$ where n is the length of the strings

```
string add(string n1, string n2)
{
    string res = "";
    int sum, carry = 0;
    for (int i = sz(n1) - 1; i >= 0; i--)
    {
        sum = n1[i] - '0' + n2[i] - '0' + carry;
        if (sum > 9)
        {
            res = to_string(sum % 10) + res;
            carry = sum / 10;
        }
        else
        {
            res = to_string(sum) + res;
            carry = 0;
        }
    }
    if (carry)
        res = to_string(carry) + res;
    return res;
}
```

stringDivisible.cpp
Description: Checks if a large number represented as a string is divisible by an integer b. Returns the remainder; if remainder is 0, the number is divisible.
Memory: $\mathcal{O}(1)$
Time: $\mathcal{O}(n)$ where n is the length of the string

```
int is_div(string a, int b)
{
    int j = 0;
    if (a[0] == '-')
        j = 1;
    if (b < 0)
        b = abs(b);
    ll rim = 0;
    for (; j < sz(a); j++)
    {
        rim = rim * 10 + (a[j] - '0');
        rim %= b;
    }
}
```

```
    }
    return rim; // if rim=0 divisible else not
}
```

stringMul.cpp
Description: Multiplies two large integers represented as strings and returns the product as a string.
Memory: $\mathcal{O}(n + m)$
Time: $\mathcal{O}(n * m)$ where n and m are lengths of n1 and n2

```
string mul(string n1, string n2)
{
    if (n1 == "0" || n2 == "0")
        return "0";
    string product(sz(n1) + sz(n2), 0);
    for (int i = sz(n1) - 1; i >= 0; i--)
    {
        for (int j = sz(n2) - 1; j >= 0; j--)
        {
            int n = (n1[i] - '0') * (n2[j] - '0') + product[i + j + 1];
            product[i + j + 1] = n % 10;
            product[i + j] += (n / 10);
        }
    }
    for (int i = 0; i < sz(product); i++)
    {
        product[i] += '0';
    }
    return product[0] == '0' ? product.substr(1) : product;
}
```

4.5 Others

CRT.cpp
Description: Chinese Remainder Theorem. crt(a, m, b, n) computes x such that $x \equiv a \pmod{m}$, $x \equiv b \pmod{n}$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

fibonacci number faster.cpp

Description: Computes the n-th Fibonacci number modulo 'mod' using the fast doubling method in $\mathcal{O}(\log n)$ time.
Usage: int val = fib(10, 100); // returns 55
Memory: $\mathcal{O}(1)$
Time: $\mathcal{O}(\log n)$

```
int fib(long long n, int mod)
{
    assert(n >= 0);
    if (n <= 1)
        return n;
    int a = 0, b = 1;
    long long i = 1ll << (63 - __builtin_clzll(n) - 1);
    for (; i; i >>= 1)
```

```
{
    int na = (a * (long long)a + b * (long long)b) % mod;
    int nb = (2ll * a + b) * b % mod;
    a = na;
    b = nb;
    if (n & 1)
    {
        int c = a + b;
        if (c >= mod)
            c -= mod;
        a = b;
        b = c;
    }
    return b;
}
```

EGCD.cpp
Description: Computes gcd(a,b) and integers x,y such that $a \cdot x + b \cdot y = \text{gcd}(a, b)$. Can be used to find modular inverses when $\text{gcd}(a, b) = 1$. Finds two integers x and y , such that $ax + by = \text{gcd}(a, b)$. If you just need gcd, use the built in _gcd instead. If a and b are coprime, then x is the inverse of $a \pmod{b}$.
Memory: $\mathcal{O}(1)$
Time: $\mathcal{O}(\log(\max(a, b)))$

```
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}
```

EGCD LDE.cpp
Description: Finds any integer solution (x_0, y_0) to the equation $a \cdot x + b \cdot y = c$, if it exists. Also computes $g = \text{gcd}(a, b)$. All solutions can then be expressed as: $x = x_0 + k \cdot \frac{b}{g}$, $y = y_0 - k \cdot \frac{a}{g}$ for integer k .
Usage: int x, y, g; if(find_any_solution(a, b, c, x, y, g)) { x, y is one solution, g = gcd(a,b) }
Memory: $\mathcal{O}(1)$
Time: $\mathcal{O}(\log(\max(a, b)))$

```
"EGCD.cpp"
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}

x0 *= c / g;
y0 *= c / g;
if (a < 0) x0 = -x0;
if (b < 0) y0 = -y0;
return true;
}
```

4.6 More Number Theory

$$\sum_{k=1}^n \frac{1}{\text{gcd}(k, n)} = \sum_{d|n} \frac{1}{d} \phi\left(\frac{n}{d}\right) = \frac{1}{n} \sum_{d|n} d \phi(d)$$
$$\sum_{k=1}^n \frac{k}{\text{gcd}(k, n)} = \frac{n}{2} \cdot \frac{1}{n} \sum_{d|n} d \phi(d)$$

$$\sum_{k=1}^n \frac{n}{\text{gcd}(k, n)} = 2 \sum_{k=1}^n \frac{k}{\text{gcd}(k, n)} - 1, \quad n > 1$$
$$\sum_{i=1}^n \sum_{j=1}^n [\text{gcd}(i, j) = 1] = \sum_{d=1}^n \mu(d) \left\lfloor \frac{n}{d} \right\rfloor^2$$
$$\sum_{i=1}^n \sum_{j=1}^n \text{gcd}(i, j) = \sum_{d=1}^n \phi(d) \left\lfloor \frac{n}{d} \right\rfloor^2$$
$$\sum_{i=1}^n \sum_{j=1}^n i \cdot j [\text{gcd}(i, j) = 1] = \sum_{i=1}^n \phi(i) i^2$$

$$\sum_{i=1}^n \sum_{j=1}^n \text{lcm}(i, j) = \sum_{l=1}^n \left(\frac{(1 + \lfloor n/l \rfloor) \lfloor n/l \rfloor}{2} \right)^2$$

4.6.1 Permutations Factorial Values

n	1	2	3	4	5	6	7	8
$n!$	1	2	6	24	120	720	5040	40320

n	9	10	11	12	13
$n!$	362880	3628800	4.0e7	4.8e8	6.2e9

n	14	15	16	17	20	25
$n!$	8.7e10	1.3e12	2.1e13	3.6e14	2e18	2e25

Graph (5)

01 BFS.cpp
Description: Finds shortest paths from source s in a graph with edge weights 0 or 1 using a deque.
Usage: Initialize distances 'd', push source 's' to deque, and run the BFS loop.
Memory: $\mathcal{O}(n + m)$
Time: $\mathcal{O}(n + m)$

```
vector<int> d(n, INF);
d[s] = 0;
deque<int> q;
q.push_front(s);
while (!q.empty()) {
    int v = q.front();
    q.pop_front();
    for (auto edge : adj[v]) {
        int u = edge.first;
        int w = edge.second;
        if (d[v] + w < d[u]) {
            d[u] = d[v] + w;
            if (w == 1)
                q.push_back(u);
            else
                q.push_front(u); // 0 gets priority
        }
    }
}
```

Cycle detection.cpp

Time: $\mathcal{O}(v + e)$

8d0a97, 40 lines

```

const long long N = 1e5 + 5;
vector<long long> v1[N];
vector<bool> visited(N, false), path(N, 0);
bool cycle = false;
void DFS(long long src)
{
    visited[src] = true;
    path[src] = true;
    for (auto it : v1[src])
    {
        if (!visited[it])
        {
            DFS(it);
        }
        else if (path[it])
            cycle = true;
    }
    path[src] = false;
}
void solve()
{
    long long n, e, u, v, i;
    cin >> n >> e;
    while (e--)
    {
        cin >> u >> v;
        // undirected
        v1[u].push_back(v);
        v1[v].push_back(u);
    }
    for (i = 0; i < n; i++)
    {
        if (!visited[i])
        {
            DFS(i);
        }
    }
    if (cycle) cout << "YES";
    else cout << "NO";
}

```

BFS path print.cpp

Time: $\mathcal{O}(v + e)$

1b2d98, 49 lines

```

vector<long long> v1[1009], parent(1009, -1);
// 1009->question a maximum node joto
// thakte pare
vector<bool> visited(1009, false);

void BFS(long long src)
{
    queue<long long> q1;
    q1.push(src);
    visited[src] = true;
    while (!q1.empty())
    {
        for (auto it : v1[q1.front()])
        {
            if (!visited[it])
            {
                q1.push(it);
                visited[it] = true;
                parent[it] = q1.front();
            }
        }
        q1.pop();
    }
}

```

```

}

void Path_print(long long vertex)
{
    if (parent[vertex] == -1)
    {
        cout << vertex << ' ';
        return;
    }
    Path_print(parent[vertex]);
    cout << vertex << ' ';
}

void solve()
{
    long long n, e, u, v, source, destination;
    cin >> n >> e;
    while (e--)
    {
        cin >> u >> v;
        // undirected
        v1[u].push_back(v);
        v1[v].push_back(u);
    }
    cin >> source >> destination;
    BFS(source);
    Path_print(destination);
}

```

Graph Coloring.cpp
Description: Checks whether a graph is bipartite using DFS by coloring each node with two colors (1 and 2). If any conflict arises, an odd cycle exists, and the graph is not bipartite.
Usage: Run dfs(startNode, 1) for every unvisited component. If flag becomes true, the graph is not bipartite.
Memory: $\mathcal{O}(N + M)$
Time: $\mathcal{O}(N + M)$ where N = nodes, M = edges

10b1ea, 22 lines

```

const int N = 200020;
vector<int> edge[200010];
int col[N];
bool flag = 0;
// colour korbo 1 or 2 diye....
void dfs(int u, int c)
{
    if (col[u]) // aghe jodi colour kora takhe
        ...
    {
        if (col[u] != c) // colour ta jodi
            kankito colour na hoi...
        {
            flag = 1; // odd length er cycle
            exist kore graph colour
            kora jabe na(!BIPARTITE).
            return;
        }
        return;
    }
    col[u] = c;
    for (int v : edge[u])
    {
        dfs(v, 3 - c); //(colour jodi 1 hoi
            amra er adjacent gula(3 - 1) == 2
            diye colour korbo)...
    } //(colour jodi 2 hoi amra er adjacent
        gula (3 - 2) == 1 diye colour korbo)
    ...
}

```

```

}

5.1 DSU and Components
DSU.cpp
Description: Implements Disjoint Set Union (DSU) with path compression and union by size. - Initializing a node u as its own set: makeset(u) - Finding the representative (root) of a set containing u: fnd(u) - Uniting sets containing u and v: unite(u, v) The main function demonstrates reading m edges and reporting whether each edge connects nodes in the same component or different components.
Memory:  $\mathcal{O}(n)$ 
Time:  $\mathcal{O}(\alpha(n))$  amortized per operation
1b71a2, 43 lines

// upobir
const int nmax = 1e5+10;

int master[nmax], sz[nmax];
void makeset(int u){
    master[u] = u;
    sz[u] = 1;
}
int fnd(int u){
    if(master[u] == u) return u;
    return master[u] = fnd(master[u]);
}
bool unite(int u, int v){
    u = fnd(u);
    v = fnd(v);
    if(u == v) return false;
    if(sz[u] < sz[v]){
        master[u] = v;
        sz[v] += sz[u];
    }
    else{
        master[v] = u;
        sz[u] += sz[v];
    }
    return true;
}

int main(){
    int n, m;
    cin >> n >> m;
    for(int i = 1; i <= n; i++) makeset(i);
    for(int i = 0; i<m; i++){
        int u, v;
        cin>>u>>v;
        bool res = unite(u, v);
        if(res){
            cout<<"edge was between two
different components"<<endl;
        }
        else{
            cout<<"edge was between same
component"<<endl;
        }
    }
}

```

Connected Comp.cpp
Description: Finds all connected components in an undirected graph with *n* vertices. Uses DFS to explore unvisited vertices and collects each component in comp.
Usage: Initialize {*adj*} and {*used*}, then call {*find comps*}() to print components.
Memory: $\mathcal{O}(n + m)$

Time: $\mathcal{O}(n + m)$

3cc69a, 27 lines

```

int n;
vector<vector<int>> adj;
vector<bool> used;
vector<int> comp;

void dfs(int v) {
    used[v] = true ;
    comp.push_back (v);
    for (int u : adj[v]) {
        if (!used[u])
            dfs (u);
    }
}

void find_comps() {
    fill(used.begin(), used.end(), 0);
    for (int v = 0; v < n; ++v) {
        if (!used[v]) {
            comp.clear();
            dfs (v);
            cout << "Component:" ;
            for (int u : comp)
                cout << ' ' << u;
            cout << endl ;
        }
    }
}

5.2 Shortest Paths
Floydwarshall.cpp
Description: Computes the maximum path weights between all pairs of vertices using Floyd-Warshall. - 'dist[u][v]' stores the maximum weight from vertex u to v. - 'link[u][v]' stores the next vertex on the path from u to v. The main function reads a graph with n vertices and m edges, updates the distance and link matrices, computes all-pairs maximum paths, prints the 'dist' matrix, and demonstrates reconstructing a path from vertex 1 to vertex n.
Memory:  $\mathcal{O}(n^2)$ 
Time:  $\mathcal{O}(n^3)$ 
565c9f, 63 lines

// upobir

const int nmax = 500;

long long int dist[nmax][nmax];
int link[nmax][nmax];

void floydwarshall(int v){
    for(int k = 1; k<=v; k++){
        for(int i = 1; i<= v; i++){
            for(int j = 1; j<=v; j++){
                if(dist[i][j] < dist[i][k] +
                    dist[k][j]){
                    dist[i][j] = dist[i][k] +
                        dist[k][j];
                    link[i][j] = link[i][k];
                }
            }
        }
    }
}

int main(){
    int n, m; cin >>n >> m;

```

```
for(int u = 1; u<=n; u++){
    for(int v = 1; v <= n; v++){
        if(u == v)
            dist[u][v] = 0;
        else
            dist[u][v] = 1e18;//LLONG_MAX;
    }

    for(int i = 0; i<m; i++){
        long long int u, v, c;
        cin>>u>>v>>c;

        dist[u][v] = min(c, dist[u][v]);
        link[u][v] = v;
    }

    floydwarshall(n);

    for(int i = 1; i<=n; i++){
        for(int j = 1; j<=n; j++){
            cout<<dist[i][j]<<" ";
        }
        cout<<endl;
    }

    int u = 1, v = n;

    cout<<u<<endl;
    while(u != v){
        u = link[u][v];
        cout<<u<<endl;
    }

    return 0;
}
```

dijkstra with path.cpp

Description: Computes shortest paths from a source vertex s in a weighted undirected graph. - $adj[u]$ stores edges as pairs $(cost, v)$. - $dist[v]$ stores the shortest distance from s to v . - $link[v]$ stores the previous vertex on the shortest path to v , allowing reconstruction of the path.

Usage: Initialize $\{adj\}$, then call $\{dijkstra\}(s, n)$ to compute distances and paths.

Memory: $\mathcal{O}(n + m)$

Time: $\mathcal{O}((n + m) \log n)$ using a priority queue

```
const int nmax = 1e5+10;
vector<pair<int, int>> adj[nmax]; //
    adj[i] = neighbour list of i
// (cost, node)
int dist[nmax];
int link[nmax];

using pii = pair<int, int>;

void dijkstra(int source, int n){
    for(int i = 1; i<=n; i++){
        dist[i] = 1e9 + 10;

        dist[source] = 0;

        priority_queue< pii , vector<pii> ,
            greater<pii> > pq;

        pq.push({0, source});
```

```
while(!pq.empty()){
    pii p = pq.top();
    pq.pop();

    int d = p.first;
    int u = p.second;

    //        cout<<"popped "<<u<<" , "<<d<<endl;

    if(dist[u] != d){
        continue;
    }

    for(auto q : adj[u]){
        int c = q.first;
        int v = q.second;

        if(d + c < dist[v]){
            dist[v] = d+c;
            link[v] = u;

            pq.push({dist[v], v});
        }
    }
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);

    int n, m;
    cin>>n>>m;

    for(int i = 0; i<m; i++){
        int u, v, w;

        cin>>u>>v>>w;

        adj[u].push_back({w, v});
        adj[v].push_back({w, u});
    }

    dijkstra(1, n);

    for(int i = 1; i<=n; i++){
        cout<<"distance to "<<i<<" = "<<dist[i]
            ]<<endl;
    }

    int current = n;

    vector<int> path;
    path.push_back(n);

    while(current != 1){
        current = link[current];
        path.push_back(current);
    }

    reverse(path.begin(), path.end());

    for(int x : path){
        cout<<x<<" ";
    }
    cout<<endl;
```

```
return 0;
}

5.3 Minimum Spanning Tree
MST prim.cpp
Description: Computes the Minimum Spanning Tree (MST) of a connected, weighted undirected graph. - 'adj[u]' stores all edges '(v, cost)' from vertex u. - 'dist[v]' stores the minimum edge weight to include v in MST. - 'completed[v]' marks whether v has been included in the MST. The function 'prim(root, n)' constructs the MST starting from vertex root.
Memory: O(V + E)
Time: O(E log V) using a priority queue      adafc0, 47 lines

const int nmax = 1e5+10; // upobir

vector<pair<int, int>> adj[nmax];

int dist[nmax];
bool completed[nmax];
void prim(int root, int n){
    for(int i = 1; i<=n; i++) dist[i] = 1e9,
        completed[i] = false;
    dist[root] = 0;

    priority_queue<pair<int, int>, vector<pair
        <int, int>>, greater<pair<int, int>>()
        > pq;

    pq.push(make_pair(0, root));

    while(!pq.empty()){
        int u = pq.top().second;
        int c = pq.top().first;

        if(completed[u]) continue;
        completed[u] = true;

        pq.pop();

        for(auto e : adj[u]){
            if(dist[e.first] > e.second){
                dist[e.first] = e.second;
                pq.push(make_pair(e.first, e.
                    second));
            }
        }
    }

    int main(){
        int V, E;
        cin>>V>>E;

        for(int i = 0; i<E; i++){
            int u, v, c;
            cin>>u>>v>>c;
            adj[u].push_back(make_pair(v, c));
            adj[v].push_back(make_pair(u, c));
        }

        prim(1, V);
    }
```

5.4 Topological Sorting

Topological Sorting.cpp

Description: Computes a topological ordering of a directed graph with n vertices. - $'g[u]'$ stores all outgoing edges from vertex u . - $'vis[u]'$ marks whether vertex u has been visited during DFS. - $'ord'$ stores the vertices in reverse postorder, giving the topological order after reversing. The code also checks for cycles: if a cycle exists, outputs "IMPOSSIBLE".

Memory: $\mathcal{O}(n + m)$

Time: $\mathcal{O}(n + m)$

```
662127, 47 lines

const int N = 1e5 + 9;
vector<int> g[N];
bool vis[N];
vector<int> ord;
void dfs(int u) {
    vis[u] = true;
    for (auto v: g[u]) {
        if (!vis[v]) {
            dfs(v);
        }
    }
    ord.push_back(u);
}

int32_t main() {

    int n, m; cin >> n >> m;
    while (m--) {
        int u, v; cin >> u >> v;
        g[u].push_back(v);
    }
    for (int i = 1; i <= n; i++) {
        if (!vis[i]) {
            dfs(i);
        }
    }
    reverse(ord.begin(), ord.end());

    // check is feasible
    vector<int> pos(n + 1);
    for (int i = 0; i < (int) ord.size(); i++) {
        pos[ord[i]] = i;
    }
    for (int u = 1; u <= n; u++) {
        for (auto v: g[u]) {
            // We need to consider self loops too!
            if (pos[u] >= pos[v]) {
                cout << "IMPOSSIBLE\n";
                return 0;
            }
        }
    }

    // print the order
    for (auto u: ord) cout << u << ' ';
    cout << '\n';
    return 0;
}
```

Geometry (6)

6.1 Geometric Basics

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)


```
template<class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
typedef Point P;
T x, y;
explicit Point(T _x=0, T _y=0) : x(_x),y(_y){}
bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
bool operator==(P p) const { return tie(x,y)== tie(p.x,p.y); }
P operator+(P p) const{return P(x+p.x,y+p.y);}
P operator-(P p) const{return P(x-p.x,y-p.y);}
P operator*(T d) const { return P(x*d, y*d); }
P operator/(T d) const { return P(x/d, y/d); }
T dot(P p) const { return x*p.x + y*p.y; }
T cross(P p) const { return x*p.y - y*p.x; }
T cross(P a, P b) const { return (a-*this).cross(b-*this); }
T dist2() const { return x*x + y*y; }
double dist() const { return sqrt((double) dist2()); }
// angle to x-axis in interval [-pi, pi]
double angle() const { return atan2(y, x); }
// makes dist() = 1
P unit() const { return *this/dist(); }
// rotate by +90 degree
P perp() const { return P(-y, x); }
P normal() const { return perp().unit(); }
//rotate 'a' radians ccw around (0,0)
P rotate(double a) const { return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
friend ostream& operator<<(ostream& os, P p) {
return os<<"("<< p.x << ", " << p.y << ")";}
};
```

lineEquation.h

Description: Gives normalized line equation $ax + by = c$, passing through points p and q.

```
tuple<ll,ll,ll> get_line_eq (Point p, Point q)
{
Point normal = (q - p).perp();
if (normal.x < 0) {
normal.x *= -1;
normal.y *= -1;
} else if (normal.x == 0 && normal.y < 0) {
normal.y *= -1;
} // ax + by = c
ll c = normal.dot(p);
ll g = gcd(c, gcd(normal.x, normal.y));
return {normal.x / g, normal.y / g, c / g;}
}
```

lineDistance.h

Description:

Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.



```
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
return (double) (b-a).cross(p-a) / (b-a).dist() ;
}
```

SegmentDistance.h

Description:

Returns the shortest distance between point p and the line segment from point s to e.



Usage: Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;

```
"Point.h"
5c88f4, 6 lines

typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
if (s==e) return (p-s).dist();
auto d = (e-s).dist2(), t = min(d,max(.0, (p-s).dot(e-s)));
return ((p-s)*d-(e-s)*t).dist()/d;
}
```

SegmentIntersection.h

Description:

If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.



Usage: vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;

```
"Point.h", "OnSegment.h"
9d57f2, 13 lines
```

```
template<class P> vector<P> segInter(P a, P b, P c, P d) {
auto oa = c.cross(d, a), ob = c.cross(d, b),
oc = a.cross(b, c), od = a.cross(b, d);
// Checks if intersection is single non-endpoint point.
if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
return {(a * ob - b * oa) / (ob - oa)};
set<P> s;
if (onSegment(c, d, a)) s.insert(a);
if (onSegment(c, d, b)) s.insert(b);
if (onSegment(a, b, c)) s.insert(c);
if (onSegment(a, b, d)) s.insert(d);
return {all(s)};
}
```

lineIntersection.h

Description:

If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.



Usage: auto res = lineInter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection point at " << res.second << endl;

```
"Point.h"
a01f81, 8 lines

template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2)
{
auto d = (e1 - s1).cross(e2 - s2);
if (d == 0) // if parallel
return {-(s1.cross(e1, s2) == 0), P(0, 0)} ;
auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
return {1, (s1 * p + e1 * q) / d};
}
```

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 ⇔ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.
Usage: bool left = sideOf(p1,p2,q)==1;

```
"Point.h"
3af81c, 9 lines

template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }
```

```
template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
auto a = (e-s).cross(p-s);
double l = (e-s).dist()*eps;
```

```
return (a > l) - (a < -l);
}
```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p) <= epsilon) instead when using Point<double>.

```
"Point.h"
c597e8, 3 lines

template<class P> bool onSegment(P s, P e, P p)
{
return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

linearTransformation.h

Description:

Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.



```
"Point.h"
03a306, 6 lines

typedef Point<double> P;
P linearTransformation(const P& p0, const P& p1, const P& q0, const P& q1, const P& r) {
P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}
```

LineProjectionReflection.h

Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

```
"Point.h"
b5562d, 5 lines

template<class P>
P lineProj(P a, P b, P p, bool refl=false) {
P v = b - a;
return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
}
```

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: vector<Angle> v = {w[0], w[0].t360()...}; // sorted
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i

```
struct Angle {
int x, y;
int t;
```

```
Angle(int x, int y, int t=0) : x(x), y(y), t
(t) {}

Angle operator-(Angle b) const { return {x-b
.x, y-b.y, t}; }

int half() const {
assert(x || y);
return y < 0 || (y == 0 && x < 0);
}

Angle t90() const { return {-y, x, t + (half
()) && x >= 0}}; }

Angle t180() const { return {-x, -y, t +
half()}; }

Angle t360() const { return {x, y, t + 1}; }
};

bool operator<(Angle a, Angle b) {
// add a.dist2() and b.dist2() to also
compare distances
return make_tuple(a.t, a.half(), a.y * (11)b
.x) <
make_tuple(b.t, b.half(), a.x * (11)b
.y);
}

// Given two points, this calculates the
smallest angle between
// them, i.e., the angle that covers the
defined line segment.
pair<Angle, Angle> segmentAngles(Angle a,
Angle b) {
if (b < a) swap(a, b);
return (b < a.t180() ?
make_pair(a, b) : make_pair(b, a.
t360()));
}

Angle operator+(Angle a, Angle b) { // point a
+ vector b
Angle r(a.x + b.x, a.y + b.y, a.t);
if (a.t180() < r) r.t--;
return r.t180() < a ? r.t360() : r;
}

Angle angleDiff(Angle a, Angle b) { // angle b
- angle a
int tu = b.t - a.t; a.t = b.t;
return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x
, tu - (b < a)};
}


```

6.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

```
"Point.h" 84d6d3, 11 lines

typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,
pair<P, P>* out) {
if (a == b) { assert(r1 != r2); return false
; }
P vec = b - a;
double d2 = vec.dist2(), sum = r1+r2, dif =
r1-r2,
p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 =
r1*r1 - p*p*d2;
if (sum*sum < d2 || dif*dif > d2) return
false;
P mid = a + vec*p, per = vec.perp() * sqrt(
fmax(0, h2) / d2);
*out = {mid + per, mid - per};
return true;
}


```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

```
"Point.h" b0153d, 13 lines

template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P
c2, double r2) {
P d = c2 - c1;
double dr = r1 - r2, d2 = d.dist2(), h2 = d2
- dr * dr;
if (d2 == 0 || h2 < 0) return {};
vector<pair<P, P>> out;
for (double sign : {-1, 1}) {
P v = (d * dr + d.perp() * sqrt(h2) * sign
) / d2;
out.push_back({c1 + v * r1, c2 + v * r2});
}
if (h2 == 0) out.pop_back();
return out;
}


```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

```
"Point.h" e0cfba, 9 lines

template<class P>
vector<P> circleLine(P c, double r, P a, P b)
{
P ab = b - a, p = a + ab * (c-a).dot(ab) /
ab.dist2();
double s = a.cross(b, c), h2 = r*r - s*s /
ab.dist2();
if (h2 < 0) return {};
if (h2 == 0) return {p};
P h = ab.unit() * sqrt(h2);
return {p - h, p + h};
}


```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.

```
"Time: O(n)
"../../content/geometry/Point.h" a1ee63, 19 lines

typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps)
{
auto tri = [&](P p, P q) {
auto r2 = r * r / 2;
P d = q - p;
auto a = d.dot(p)/d.dist2(), b = (p.dist2
()-r*r)/d.dist2();
auto det = a * a - b;
if (det <= 0) return arg(p, q) * r2;
auto s = max(0., -a-sqrt(det)), t = min
(1., -a+sqrt(det));
}


```

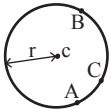
```
if (t < 0 || 1 <= s) return arg(p, q) * r2
;
P u = p + d * s, v = p + d * t;
return arg(p,u) * r2 + u.cross(v)/2 + arg(
v,q) * r2;
};
auto sum = 0.0;
rep(i,0,sz(ps))
sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)]
- c);
return sum;
}


```

circumcircle.h

Description:

The circumcirle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



```
"Point.h" 1caa3a, 9 lines

typedef Point<double> P;
double ccRadius(const P& A, const P& B, const
P& C) {
return (B-A).dist()*(C-B).dist()*(A-C).dist
())/
abs((B-A).cross(C-A))/2;
}

P ccCenter(const P& A, const P& B, const P& C)
{
P b = C-A, c = B-A;
return A + (b*c.dist2()-c*b.dist2()).perp()/
b.cross(c)/2;
}


```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.

```
"Time: expected O(n)
"circumcircle.h" 09dd0a, 17 lines

pair<P, double> mec(vector<P> ps) {
shuffle(all(ps), mt19937(time(0)));
P o = ps[0];
double r = 0, EPS = 1 + 1e-8;
rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r *
EPS) {
o = ps[i], r = 0;
rep(j,0,i) if ((o - ps[j]).dist() > r *
EPS) {
EPS) {
o = (ps[i] + ps[j]) / 2;
r = (o - ps[i]).dist();
rep(k,0,j) if ((o - ps[k]).dist() > r *
EPS) {
o = ccCenter(ps[i], ps[j], ps[k]);
r = (o - ps[i]).dist();
}
}
}
return {o, r};
}


```

6.3 Misc. Point Set

Problems

ClosestPair.h

Description: Finds the closest pair of points.
Time: $\mathcal{O}(n \log n)$

```
"Point.h" ac41a6, 17 lines

typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
assert(sz(v) > 1);
set<P> S;
sort(all(v), [](P a, P b) { return a.y < b.y
; });
pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P
()}};
int j = 0;
for (P p : v) {
P d[1 + (11)sqrt(ret.first), 0];
while (v[j].y <= p.y - d.x) S.erase(v[j
++]);
auto lo = S.lower_bound(p - d), hi = S.
upper_bound(p + d);
for (; lo != hi; ++lo)
ret = min(ret, {(*lo - p).dist2(), {*lo,
p}});
S.insert(p);
}
return ret.second;
}


```

Basics and Misc (7)

7.1 Basics.

Prefix Sum 2D.cpp

Description: Computes a 2D prefix-sum array in $\mathcal{O}(nm)$. For a matrix a , the prefix matrix $pref$ satisfies:

$$pref[i][j] = \sum_{x=1}^i \sum_{y=1}^j a[x][y].$$

A sub-rectangle sum (x_1, y_1) to (x_2, y_2) is:

$$pref[x_2][y_2] - pref[x_1 - 1][y_2] - pref[x_2][y_1 - 1] + pref[x_1 - 1][y_1]$$

Usage: Build $pref[][]$ once, then answer each query in $\mathcal{O}(1)$.

Time: $\mathcal{O}(nm)$ build, $\mathcal{O}(1)$ per query. f2c91f, 30 lines

```
const int N = 1005;
int a[N][N], pref[N][N]; // ykw
int32_t main() {
ios_base::sync_with_stdio(0);
cin.tie(0);
int n, m; cin >> n >> m;
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= m; j++) {
cin >> a[i][j];
}
}
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= m; j++) {
pref[i][j] = pref[i - 1][j] + pref[i][j
- 1] - pref[i - 1][j - 1] + a[i][j];
}
}
int q; cin >> q;
while (q--) {
int x1, y1, x2, y2; cin >> x1 >> y1 >> x2
>> y2;
}


```

```
int ans = 0;
// for (int i = x1; i <= x2; i++) {
//     for (int j = y1; j <= y2; j++) {
//         ans += a[i][j];
//     }
// }
ans = pref[x2][y2] - pref[x1 - 1][y2] -
pref[x2][y1 - 1] + pref[x1 - 1][y1 - 1];
cout << ans << '\n';
}
return 0;
}
```

Bitmask and bitwise.cpp

Description: Utilities for bitmask enumeration and bit operations. Enumerating all subsets of k elements takes 2^k iterations. Bit operations use 0-based indexing:
Usage: for (mask = 0; mask < (1 << k); mask++) iterate subsets.

Time: $\mathcal{O}(k2^k)$ for enumeration, $\mathcal{O}(1)$ for each bit op.

```
//getting all possible subset of k elements(k
elements has 2^k subsets )
void bitmask(int k) //! O(k*(2^k))
{
    for (int j = 0; j < (1 << k); j++) {
        for (int i = 0; i < k; i++) {
            if (j & (1 << i)) {
                /** take the i-th element
            }
        }
    }
}
```

```
//bitwise basics 0 - based index
#define cnt_bits(x) __builtin_popcountll(x)
#define clz(x) __builtin_clzll(x)
#define ctz(x) __builtin_ctzll(x)
```

```
ll set0(ll n, ll k){
    n = (n | (1LL << k)) ^ (1LL << k);
    return n;
}

ll set1(ll n, ll k){
    n = (n | (1LL << k));
    return n;
}

ll tog1(ll n, ll k){
    return (n ^ (1LL << k));
}

bool is_set(ll n, ll k){
    return (n & (1LL << k));
}
```

```
bool is_power_of_2(ll n){
    return (n>0 && !(n&(n-1)));
}
```

compare.cpp

Description: Two comparison helpers. Pair comparator: sorts by first element ascending; if equal, sorts by second element descending. Formally: $(A < B)$ iff $(A.first < B.first)$ or $(A.first = B.first \wedge A.second > B.second)$. Double comparator: compares two floating-point numbers with epsilon. Returns 0 if $|a - b| \leq \epsilon$, otherwise -1 if $a < b$, and 1 if $a > b$.

Usage: sort(v.begin(), v.end(), cmp);
if (double_comp(x, y) == 0) { equal }

Time: $\mathcal{O}(1)$

```
bool cmp(pair<int, int> A, pair<int, int> B)
    // pair compare
{
    if (A.first == B.first)
        return (A.second > B.second);
    //! if A.second is Larger than A will come before B
    return (A.first < B.first);
    //! if A.first is smaller than A will come before B
}

int double_comp(ld a, ld b) //compare between 2 double
{
    if (fabs(a - b) <= eps)
        return 0;
    return a < b ? -1 : 1;
    //! dbl cmp return 0 when a==b, -1 if a<b, 1 if a>b
}
```

Direction Array.cpp

Description: Direction vectors for grid and knight moves. $-4/8$ directions $(dx[i], dy[i])$, $i = 0..7$: $(+1, 0) = \text{Down}$, $(-1, 0) = \text{Up}$, $(0, +1) = \text{Right}$, $(0, -1) = \text{Left}$, $(+1, +1) = \text{Down-Right}$, $(+1, -1) = \text{Down-Left}$, $(-1, +1) = \text{Up-Right}$, $(-1, -1) = \text{Up-Left}$ - Knight moves $(kdx[i], kdy[i])$, $i = 0..7$

Time: $\mathcal{O}(1)$

```
// (dx, dy for grid-based problems) 4
directions and 4 diagonal moves
const ll dx[] = {1, -1, 0, 0, 1, 1, -1, -1};
const ll dy[] = {0, 0, 1, -1, 1, -1, 1, -1};

// Knight's Move
ll kdx[] = {-2, -2, -1, -1, 1, 1, 2, 2};
ll kdy[] = {-1, 1, -2, 2, -2, 2, -1, 1};
```

Maximum subarray sum.cpp

Memory: $\mathcal{O}(N)$

Time: $\mathcal{O}(N)$

```
ll kadane_algo(const vector<ll> & v, ll n) {
    ll temp = 0, res = 0;
    for (ll i = 0; i < n; i++) {
        temp += v[i];
        if (temp < 0) temp = 0;
        res = max(res, temp);
    }
    return res;
}
```

7.2 Misc.

Dates.h

Description: Provides functions to convert between Gregorian calendar dates and Julian Day numbers. - 'dateToInt(y, m, d)': Converts a date (year, month, day) to its corresponding Julian Day number. - 'intToDate(jd, y, m, d)': Converts a Julian Day number back to a Gregorian date. - 'intToDay(jd)': Returns the day of the week (0 = monday, 5 = Saturday) from a Julian Day number.

Usage: int jd = dateToInt(2025, 11, 14);

int y, m, d;
intToDate(jd, y, m, d);
int weekday = intToDay(jd);

Memory: $\mathcal{O}(1)$ for all functions

Time: $\mathcal{O}(1)$ for all functions

```
int intToDay(int jd) { return jd % 7; }
int dateToInt(int y, int m, int d) {
    return 1461 * (y + 4800 + (m - 14) / 12) / 4
        +
        367 * (m - 2 - (m - 14) / 12 * 12) / 12 -
        3 * ((y + 4900 + (m - 14) / 12) / 100) / 4
        +
        d - 32075;
}

void intToDate(int jd, int &y, int &m, int &d)
{
    int x, n, i, j;
    x = jd + 68569;
    n = 4 * x / 146097;
    x -= (146097 * n + 3) / 4;
    i = (4000 * (x + 1)) / 1461001;
    x -= 1461 * i / 4 - 31;
    j = 80 * x / 2447;
    d = x - 2447 * j / 80;
    x = j / 11;
    m = j + 2 - 12 * x;
    y = 100 * (n - 49) + i + x;
}
```

Random.h

Description: gen_random(l, r); random number dei l to r
er modde, shuffle() vector ke randomly suffle kore

```
mt19937_64 rng(chrono::steady_clock::now().
time_since_epoch().count());
inline ll gen_random(ll l, ll r){
return uniform_int_distribution<ll>(l, r)(rng)
;
}
```

shuffle(all(v), rng);

LIS.h

Description: Compute indices for the longest increasing subsequence.

Time: $\mathcal{O}(N \log N)$

```
template<class I> vi lis(const vector<I>& S) {
    if (S.empty()) return {};
    vi prev(sz(S));
    typedef pair<I, int> p;
    vector<p> res;
    rep(i, 0, sz(S)) {
        // change 0 -> i for longest non-
decreasing subsequence
        auto it = lower_bound(all(res), p{S[i], 0})
;
        if (it == res.end()) res.emplace_back(),
            it = res.end()-1;
        *it = {S[i], i};
        prev[i] = it == res.begin() ? 0 : (it-1)->
second;
    }
    int L = sz(res), cur = res.back().second;
    vi ans(L);
    while (L-->) ans[L] = cur, cur = prev[cur];
    return ans;
}
```

Full template (8)

8.1 Arif's template

Arif template.cpp

Description: For each position in a string, computes $p[0][i] = \text{half length of longest even palindrome around pos } i$, $p[1][i] = \text{longest odd (half rounded down)}$.

Usage: rotate(v.begin(), v.begin()+minRotation(v), v.end());

Time: $\mathcal{O}(N)$

```
<bits/stdc++.h>, <ext/pb.ds/assoc_container.hpp>,
<ext/pb.ds/tree_policy.hpp>
76280c, 1283 lines
```

using namespace std;

#define TT template <typename T

TT, typename=void> struct cerrok:false_type {};
TT> struct cerrok <T, void_t<decltype(cerr << decltypeval<T>()) >>> : true_type {};

TT> constexpr void p1 (const T &x);
TT, typename V> void p1(const pair<T, V> &x) {
 cerr << "{"; p1(x.first); cerr << ",";
 p1(x.second); cerr << "}";
}

TT> constexpr void p1 (const T &x) {
 if constexpr (cerrok<T>::value) cerr << x;
 else { int f = 0; cerr << '{';
 for (auto &i: x)
 cerr << (f++ ? "," : ""), p1(i);
 cerr << "}";
 }
}

void p2() { cerr << "]\n"; }
TT, typename... V> void p2(T t, V... v) {
 p1(t);
 if (sizeof...(v)) cerr << ",";
 p2(v...);
}

#ifndef ONLINE_JUDGE
#define dbg(x...) {cerr<<__LINE__<< "[" << #x << "]"=""; p2(x); cerr << "\e ";}
#else
#define dbg(...)
#endif

typedef long long ll;
template<class T> using V=vector<T>;
#define all(x) (x).begin(), (x).end()

#define rall(x) (x).rbegin(), (x).rend()
#define ff first
#define ss second
#define pb push_back
typedef pair<int, int> pii;
typedef vector<int> vi;
#define sz(n) int((n).size())
#define forA(cn) for(auto &x : (cn))
#define rep(i,r) for(int i=0 ; i <(r); i++)
#define repl(i,l,r) for(int i=(l); i<(r); i++)
#define repr(i,r,l) for(int i=(r); i>=(l); i--)

```
// -----BIT-----
/*
* Description: \texttt{update(i,x): a[i] += x
;}\
* \texttt{query(i): sum in [0, i);}\

```

```

* \texttt{lower\_bound}(sum): min pos st sum
of [0, pos] >= sum, returns n if all <
sum, or -1 if empty sum.
*/
struct FT {
    int n; V<ll> s;
    FT(int _n) : n(_n), s(_n) {}
    void update(int i, ll x) {
        for (; i < n; i |= i + 1) s[i] += x; }
    ll query(int i, ll r = 0) {
        for (; i > 0; i &= i - 1) r += s[i-1];
        return r; }
    int lower_bound(ll sum) {
        if (sum <= 0) return -1; int pos = 0;
        for (int pw = 1 << (31 - __builtin_clz(n))
             ; pw; pw >>= 1) {
            if (pos+pw <= n && s[pos + pw-1] < sum)
                pos += pw, sum -= s[pos-1];
        }
        return pos;
    }
};

FT f1(n), f2(n);
// a[l...r] += v; 0 <= l <= r < n
auto upd = [&](int l, int r, ll v) {
    f1.update(l, v), f1.update(r + 1, -v);
    f2.update(l, v*(l-1)), f2.update(r+1, -v
        *r);
}; // a[l] + ... + a[r]; 0 <= l <= r < n
auto sum = [&](int l, int r) { ++r;
    ll sub = f1.query(l) * (l-1) - f2.query(
        l);
    ll add = f1.query(r) * (r-1) - f2.query(
        r);
    return add - sub;
};

// 2D bit with update
template <class T=ll>
struct FT2D {
    vector<vector<T>> x;
    FT2D(int n, int m) : x(n, vector<T>(m)) {}
    void add(int k1, int k2, int a) { // x[k1][
        k2] += a
        for (; k1 < x.size(); k1 |= k1+1)
            for (int k=k2; k < x[k1].size(); k |= k
                +1) x[k1][k] += a;
    }
    T sum(int k1, int k2, T s=0) { // return x
        [0][0] to x[k1][k2] sum
        for (; k1 >= 0; k1 = (k1&(k1+1))-1)
            for (int k=k2; k >= 0; k = (k&(k+1))-1)
                s += x[k1][k];
        return s;
    }
};
// sum of rectangle [(x1, y1), (x2, y2)] 0
base
T rsum(int x1, int y1, int x2, int y2) {
    return sum(x2, y2) - (x1 ? sum(x1 - 1,
        y2) : 0) - (y1 ? sum(x2, y1 - 1)
            : 0)
        + (x1 && y1 ? sum(x1 - 1, y1 - 1)
            : 0);
}
};
//_____

```

```

// _____ data _____
struct Dsu{
    vi pa,sz;
    Dsu(int n){
        pa.resize(n);
        sz=vi(n,1);
        iota(pa.begin(),pa.end(),0);
    }
    int get(int a){
        return pa[a]=((pa[a]==a)?a:get(pa[a]));
    }
    bool add(int a,int b){
        a=get(a),b=get(b);
        if(a!=b){
            if(sz[a]<sz[b])swap(a,b);
            sz[a]+=sz[b]; pa[b]=a;
            return true;
        }
        else return false;
    }
}
};
//_____
struct Trie{
    const int B=30;
    struct node{
        node *nxt[2] ;
        int sz;
        node(){nxt[0]=nxt[1]=NULL ;sz=0 ;}
    }*root;
    Trie (){ root=new node(); }
    void insert(int x){
        node *cur= root ;
        cur->sz++ ;
        for(int i=B;i>=0;i--){
            int b=x>>i&1;
            if(cur->nxt[b]==NULL) cur->nxt[b]=new
                node() ,cur=cur->nxt[b];
            else cur=cur->nxt[b] ;
            cur->sz++;
        }
    }
    ll get_max(int x ){
        node *cur= root ;
        ll ans =0 ;
        for(int i=B;i>=0;i--){
            int b=x>>i&1;
            if(cur->nxt[b] && cur->nxt[!b]->sz
                >0) cur=cur->nxt[!b] ,ans+=(1<<i)
                ;
            else cur=cur->nxt[b] ;
        }
        return ans ;
    }
    int query(int x, int k) { // number of
        values s.t. val ^ x < k
        node* cur = root;
        int ans = 0;
        for (int i = B ; i >= 0; i--) {
            if (cur == NULL) break;
            int b1 = x >> i & 1, b2 = k >> i & 1;
            if (b2 == 1) {
                if (cur -> nxt[b1]) ans += cur -> nxt[
                    b1] -> sz;
                cur = cur -> nxt[!b1];
            } else cur = cur -> nxt[b1];
        }
    }
}

```

```

        return ans;
    }
    void remove(int val) {
        node* cur = root;
        cur -> sz--;
        for (int i = B; i >= 0; i--) {
            int b = val >> i & 1;
            cur = cur -> nxt[b];
            cur -> sz--;
        }
    }
    void del(node* cur) {
        for (int i=0;i<2;i++) if(cur->nxt[i])del(
            cur->nxt[i]);
        delete(cur);
    }
};
// basis
vector<int > bit(32) ;
int Sz=0 ;
void Insert(int x){
    for(int i=30;i>=0 ;i--){
        if((x>>i & 1) == 0 )continue ;
        if(bit[i] ) x^=bit[i] ;
        else {
            bit[i]=x ;
            Sz++ ;
            break ;
        }
    }
}
//_____
using TY= int ;
using Mat=vector<vector<TY>> ;
Mat operator*(const Mat& a, const Mat& b) {
    if (a.empty() || b.empty())return {{}};
    Mat c(sz(a) , V<TY>(sz(b[0])) );
    rep(i,sz(c)){
        rep(j,sz(c[0])) {
            c[i][j] = 0;
            rep(k,sz(b)) {
                c[i][j] += a[i][k] * b[k][j];
            }
        }
    }
    return c;
}
Mat &operator*=(Mat &a, const Mat& b) {
    return a = a * b;
}
Mat power(Mat a, ll b) {
    assert(b >= 0);
    Mat res(sz(a),V<TY>(sz(a[0])) );
    rep(i,sz(a))res[i][i] = 1;
    for(;b;a*=a,b>>=1)if(b&1)res*=a;
    return res;
}
//_____
V<V<ll>> PrefixSums2D(V<V<ll>> &a) {

```

```

// ...1 base
ll n = a.size(),m = a[0].size();
V<V<ll>> prefixSum(n + 1, vl(m + 1, 0));
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        prefixSum[i + 1][j + 1] =
            prefixSum[i][j + 1] +
            prefixSum[i + 1][j] -
            prefixSum[i][j] + a[i][j];
    }
}
return prefixSum;
}
ll get(int rx,int ry,int lx,int ly,vvl &pre){
    // ... 1 base l to r inclusive
    lx--,--ly;
    return pre[rx][ry]+pre[lx][ly]-pre[rx][ly]-
        pre[lx][ry];
}
/* pbds
using namespace __gnu_pbds;
template<class T>using ordered_set=tree<T,
    null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
template<class T>using ordered_multiset=tree<T
    , null_type, less_equal<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
*/
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}
// for many solution (x0 = x0+ k*b/g ) and (
y0 = y0- k*a/g) k is a integer
bool find_any_solution(int a, int b, int c,
    int &x0, int &y0, int &g) {
    g = euclid(abs(a), abs(b), x0, y0);
    if (c % g) return false;
    x0 *= c / g; y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no
    solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}
const ll mod = 1e9+7; // change to something
else
struct Mint {
    ll x;
    Mint() : x(0){}
    Mint(ll x_) : x(x_ % mod){if (x < 0) x += mod;
    }
    Mint operator+(Mint b) { return Mint((x + b.
        x) % mod); }
    Mint operator-(Mint b) { return Mint((x - b.
        x + mod) % mod); }
}

```

```

    Mint operator*(Mint b) { return Mint((x * b.
        x) % mod); }
    Mint operator/(Mint b) { return *this *
        invert(b); }
    Mint invert(Mint a) {
        ll x, y, g = euclid(a.x, mod, x, y);
        assert(g == 1); return Mint((x + mod) %
            mod);
    }
    Mint power(ll e) {
        if (!e) return Mint(1);
        Mint r = this->power(e / 2); r = r * r;
        return e&1 ? *this * r : r;
    }
};

vector<Mint> fact(1, 1), inv_fact(1, 1);

Mint C(int n, int k) {
    if (k < 0 || k > n) return 0;

    while (sz(fact) < n + 1) {
        fact.pb(fact.back() * sz(fact));
        inv_fact.pb(Mint(1) / fact.back());
    }
    return fact[n] * inv_fact[k] * inv_fact[n -
        k];
}

// fast factor

// works in sqrt(sqrt(n))
typedef unsigned long ull;
ull modmul(ull a, ull b, ull M) {
    ll ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (1l
        )M);
}

ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;
    for (; e; b = modmul(b, b, mod), e /= 2)
        if (e & 1) ans = modmul(ans, b, mod);
    return ans;
}

bool isPrime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1)
        == 3;
    ull A[] = {2, 325, 9375, 28178, 450775,
        9780504, 1795265022},
        s = __builtin_ctzll(n-1), d = n >> s;
    for (ull a : A) { // ^ count trailing
        zeroes
        ull p = modpow(a%n, d, n), i = s;
        while (p != 1 && p != n-1 && a % n && i
            --)
            p = modmul(p, p, n);
        if (p != n-1 && i != s) return 0;
    }
    return 1;
}

ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x, n)
        + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);

```

```

        if ((q = modmul(prd, max(x, y) - min(x, y),
            n)) prd = q;
        x = f(x), y = f(y));
    }
    return gcd(prd, n);
}

vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}

// ----- graph -----

template<class T> struct RMQ {
    V<V<T>> jmp;
    RMQ(const V<T>& V) : jmp(1, V) {
        for (int pw = 1, k = 1; pw * 2 <= sz(V);
            pw *= 2, ++k) {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            rep(j, sz(jmp[k]))
                jmp[k][j] = min(jmp[k-1][j], jmp[k-
                    1][j+pw]);
        }
    }
    T query(int a, int b) { // [a,b)
        assert(a < b); // or return inf if a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 <<
            dep)]);
    }
};

struct LCA {
    int T = 0;
    vi time, path, ret, tour;
    RMQ<int> rmq;
    LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C
        ), 0, -1), ret) {}
    void dfs(vector<vi>& C, int v, int par) {
        time[v] = T++; tour.pb(v);
        for (int y : C[v]) if (y != par) {
            path.pb(v), ret.pb(time[v]);
            dfs(C, y, v);
        }
        tour.pb(v);
    }

    int lca(int a, int b) {
        if (a == b) return a;
        tie(a, b) = minmax(time[a], time[b]);
        return path[rmq.query(a, b)];
    }
    //dist(a,b){return depth[a] + depth[b] - 2*
        depth[lca(a,b)];}
};

vector<vi> treeJump(vi& P) {
    int on = 1, d = 1;
    while(on < sz(P)) on *= 2, d++;
    vector<vi> jmp(d, P);
    repl(i, 1, d) repl(j, 0, sz(P))
        jmp[i][j] = jmp[i-1][jmp[i-1][j]];
    return jmp;
}

int jmp(vector<vi>& tbl, int nod, int steps) {
    repl(i, 0, sz(tbl))

```

```

        if(steps & (1<<i)) nod = tbl[i][nod];
        return nod;
    }
}

int lca(vector<vi>& tbl, vi& depth, int a, int
    b) {
    if (depth[a] < depth[b]) swap(a, b);
    a = jmp(tbl, a, depth[a] - depth[b]);
    if (a == b) return a;
    for (int i = sz(tbl); i--;) {
        int c = tbl[i][a], d = tbl[i][b];
        if (c != d) a = c, b = d;
    }
    return tbl[0][a];
}

// art bri
int n, timer;
V<vi> adj;
V<pii> bri;
vi vis, tin, Sz, low, art;
void dfs(int v, int p = -1) {
    vis[v] = true;
    Sz[v] = 1;
    tin[v] = low[v] = timer++;
    int child = 0;
    bool pas = false;
    for (int to : adj[v]) {
        if (to == p && !pas) {
            pas = true;
            continue;
        }
        if (vis[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            Sz[v] += Sz[to];
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1) art
                [v] = 1;
            if (low[to] > tin[v]) bri.pb({v, to
                });
            ++child;
        }
        if (p == -1 && child > 1) art[v] = 1;
    }
}

void art_bri() {
    timer = 0; bri.clear();
    Sz = vi(n, 0);
    vis = vi(n, 0); art = vi(n, 0);
    tin = vi(n, -1); low = vi(n, -1);
    rep(i, n) {
        if (!vis[i]) dfs(i);
    }
}

pair<vi, vi> Fun(const V<vi> &g, const vi &
    order) {
    vi topo, vis(sz(g)), was(sz(g));
    int attempt = 0;
    auto dfs = [&](auto dfs, int v) -> void {
        vis[v] = 1;
        was[v] = attempt;
        for (auto u : g[v]) {
            if (vis[u] == 0)

```

```

                dfs(dfs, u);
            }
            topo.pb(v);
        };
        rep(i, sz(order)) {
            if (vis[order[i]] == 0) {
                attempt++;
                dfs(dfs, order[i]);
            }
        }
        reverse(all(topo));
        return make_pair(topo, was);
    }
    // return SCC
    vi SCC(const V<vi> &g, V<vi>&rg) {
        vi ord(sz(g)); iota(all(ord), 0);
        auto [topo, _] = Fun(g, ord);
        auto [__, was] = Fun(rg, topo); // was has
            scc with same val
        return was;
    }

    vi topoSort(const vector<vi>& gr) {
        vi indeg(sz(gr)), q;
        for (auto& li : gr) for (int x : li) indeg[x]
            ++;
        rep(i, sz(gr)) if (indeg[i] == 0) q.pb(i);
        rep(j, sz(q)) for (int x : gr[q[j]])
            if (--indeg[x] == 0) q.pb(x);
        return q;
    }

    /*
    // Floyd Warshall
    int n, m;

    vector<vector<ll>> dis(n, vector<ll>(n, inf));

    for (int i = 0; i < n; i++) dis[i][i] = 0;

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dis[i][k] != inf && dis[k][j] != inf)
                    dis[i][j] = min(dis[i][j], dis[i][k] +
                        dis[k][j]);
            }
        }
    }
    */
    // adj list with edges ids
    vi eulerWalk(vector<vector<pii>>& gr, int
        nedges, int src = 0) {
        int n = sz(gr);
        vi D(n), its(n), eu(nedges), ret, s = {src};
        D[src]++; // to allow Euler paths, not just
            cycles
        while (!s.empty()) {
            int x = s.back(), y, e, &it = its[x], end
                = sz(gr[x]);
            if (it == end) { ret.push_back(x); s.
                pop_back(); continue; }
            tie(y, e) = gr[x][it++];
            if (!eu[e]) {
                D[x]--; D[y]++;
                eu[e] = 1; s.push_back(y);
            }
        }
    }
}

```



```

    for (int x : D) if (x < 0 || sz(ret) !=
        nedges+1) return {};
    return {ret.rbegin(), ret.rend()};
}

template <class T> int sgn(T x) { return (x >
    0) - (x < 0); }
const double eps = 1e-9;
const double PI = acos((double)-1.0);
// int sgn(double x) { return (x > eps) - (x <
    -eps); }

template<class T> struct Point {
typedef Point P; T x, y;
explicit Point(T _x=0, T _y=0) : x(_x), y(_y){}
bool operator<(P p) const { return tie(x,y) <
    tie(p.x,p.y); }
bool operator==(P p) const { return tie(x,y)==
    tie(p.x,p.y); }
P operator+(P p) const{return P(x+p.x,y+p.y);}
P operator-(P p) const{return P(x-p.x,y-p.y);}
P operator*(T d) const { return P(x*d, y*d); }
P operator/(T d) const { return P(x/d, y/d); }
T dot(P p) const { return x*p.x + y*p.y; }
T cross(P p) const { return x*p.y - y*p.x; }
T cross(P a, P b) const { return (a-*this).
    cross(b-*this); }
T dist2() const { return x*x + y*y; }
double dist() const { return sqrt((double)
    dist2()); }
// angle to x-axis in interval [-pi, pi]
double angle() const { return atan2(y, x); }
// makes dist() = 1
P unit() const { return *this/dist(); }
// rotate by +90 degree
P perp() const { return P(-y, x); }
P normal() const { return perp().unit(); }
//rotate 'a' radians ccw around (0,0)
P rotate(double a) const { return P(x*cos(a)-y
    *sin(a),x*sin(a)+y*cos(a)); }
friend ostream& operator<<(ostream& os, P p) {
    return os<<"("<< p.x << ", " << p.y << ")";}
friend istream &operator>>(istream &in,P &p){
    return in >>p.x>>p.y; }
};
// a—b seg [cl,0] [-1, cw] [1 ,ccw]
template<class P>
int orientation(P a, P b, P c) { return sgn(a.
    cross(b,c)); }

double rad_to_deg(double r) { return (r *
    180.0 / PI); }
double deg_to_rad(double d) { return (d * PI /
    180.0); }

template<class P> double get_angle(P a, P b)
{
    double costheta = a.dot(b) / a.dist() / b.
        dist();
    return acos(max((double)-1.0, min((double)
        1.0, costheta)));
}
template<class P>
bool is_point_in_angle(P b, P a, P c, P p) {
    // does point p lie in angle <bac
    assert(orientation(a, b, c) != 0);
    if (orientation(a, c, b) < 0) swap(b, c);
    return orientation(a, c, p) >= 0 &&
        orientation(a, b, p) <= 0;
}

```

```

}
// checks if point p in s— e line segment
template<class P>
bool onSegment(P s, P e, P p) {return p.cross(
    s, e) == 0 && (s - p).dot(e - p) <= 0;}
template<class P>
vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d,
        b),
        oc = a.cross(b, c), od = a.cross(b,
        d);
    // Checks if intersection is single non-
        endpoint point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) *
        sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa
            )};
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
    return {all(s)};
}
//Returns the shortest distance between point
    p and the line segment from point s to e.
// typedef Point<double> P;
// double segDist(P s, P e, P p) {
//     if (s==e) return (p-s).dist();
//     auto d = (e-s).dist2(), t = min(d,max
//         (.0,(p-s).dot(e-s)));
//     return ((p-s)*d-(e-s)*t).dist()/d;
// }
// [-1 for boundary ][1 for inside ] else
    outside
template<class P>
int P_i_polygon(vector<P>&p ,P &x){
    P y = P(x.x + 1, 1l(1e9)+711);
    int cnt =0 ,n=sz(p) ;
    rep(i,n){
        cnt+=sz(segInter(p[i] ,p[(i+1)%n]
            ,x,y) )>0 ;
        if(onSegment(p[i], p[ (i+1)%n], x)
            )return -1 ;
        if(cnt&1) return 1;
        else return 0 ;
    }
}
template< class P>
vector<P> convex_hull(vector<P> &p) {
    if (p.size() <= 1) return p;
    vector<P> v = p;
    sort(v.begin(), v.end());
    vector<P> up, dn;
    for (auto& p : v) {
        while (up.size() > 1 && orientation(up
            [up.size() - 2], up.back(), p) >=
            0) {
            up.pop_back();
        }
        while (dn.size() > 1 && orientation(dn
            [dn.size() - 2], dn.back(), p) <=
            0) {
            dn.pop_back();
        }
        up.pb(p); dn.pb(p);
    }
}

```

```

v = dn;
if (v.size() > 1) v.pop_back();
reverse(up.begin(), up.end());
up.pop_back();
for (auto& p : up) v.pb(p);

if (v.size() == 2 && v[0] == v[1]) v.
    pop_back();
return v;
}

// —string —
// Description: Static hashing for 0-indexed
    string. Intervals are $[l, r]$.
template<const ll M, const ll B>
struct Hashing {
    int n; V<ll> h, pw;
    Hashing(const string &s) : n(sz(s)),h(n+1),
        pw(n+1) {
        pw[0] = 1; // ^^ s is 0 indexed
        for (int i = 1; i <= n; ++i)
            pw[i] = (pw[i-1] * B) % M,
            h[i] = (h[i-1] * B + s[i-1]) % M;
    }
    ll eval(int l, int r) { // assert(l <= r);
        return (h[r+1] - ((h[l] * pw[r-l+1])%M) +
            M)%M;
    }
};
struct Double_Hash {
    using H1 = Hashing<916969619, 101>;
    using H2 = Hashing<285646799, 103>;
    H1 h1; H2 h2;
    Double_Hash(const string &s):h1(s),h2(s){}
    pii eval(int l, int r)
        { return {h1.eval(l,r), h2.eval(l,r)}; }
};

// Description: Hashing with point updates on
    string (0-indexed).
// \texttt{upd(i, x): s[i] += x}. Intervals
    are $[l, r]$.
// update delta = new c- s[pos ];

template<const ll M, const ll B>
struct Dynamic_Hashing {
    int n; V<ll> h, pw;
    void upd(int pos, int c_add) {
        if (c_add < 0) c_add = (c_add + M) % M;
        for (int i = ++pos; i <= n; i += i&-i)
            h[i] = (h[i]+c_add *1LL* pw[i - pos])%M;
    }
    ll get(int pos, int r = 0) {
        for (int i = ++pos, j = 0; i; i -= i&-i) {
            r = (r + h[i] * 1LL * pw[j]) % M;
            j += i&-i;
        } return r;
    }
}
Dynamic_Hashing(const string &s) : n(sz(s)),
    h(n+1), pw(n+1) {
    pw[0] = 1; // ^^ s is 0 indexed
    for (int i = 1; i <= n; ++i) pw[i] = (pw[i
        -1] * 1LL * B) % M;
    for (int i = 0; i < n; ++i) upd(i, s[i]);
}
ll eval(int l, int r) { // assert(l <= r);
    return (get(r) - ((get(l-1) * 1LL * pw[r-l
        +1]) % M) + M) % M;
}
};
struct Double_Dynamic {

```

```

    using DH1 = Dynamic_Hashing<916969619, 571>;
    using DH2 = Dynamic_Hashing<285646799, 953>;
    DH1 h1; DH2 h2;
    Double_Dynamic(const string &s) : h1(s), h2(
        s) {}
    void upd(int pos, int c_add) {
        h1.upd(pos, c_add);
        h2.upd(pos, c_add);
    }
    pll eval(int l, int r)
        { return {h1.eval(l,r), h2.eval(l,r)}; }
};

/* * Description: pi[x] computes the length of
    the longest prefix of s that ends at x,
    * other than s[0...x] itself (abacaba ->
        0010123).
    * Can be used to find all occurrences of a
        string. */
vi pi(const string& s) {
    vi p(sz(s));
    repl(i,1,sz(s)) {
        int g = p[i-1];
        while (g && s[i] != s[g]) g = p[g-1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}

vi match(const string& s, const string& pat) {
    vi p = pi(pat + '\0' + s), res;
    repl(i,sz(p)-sz(s),sz(p))
        if (p[i] == sz(pat)) res.push_back(i - 2 *
            sz(pat));
    return res;
}

/*
    * Description: z[i] computes the length of
        the longest common prefix of s[i:] and s,
    * except z[0] = 0. (abacaba -> 0010301)
    */
vi Z(const string& S) {
    vi z(sz(S));
    int l = -1, r = -1;
    repl(i,1,sz(S)) {
        z[i] = i >= r ? 0 : min(r - i, z[i - 1]);
        while (i + z[i] < sz(S) && S[i + z[i]] ==
            S[z[i]])
            z[i]++;
        if (i + z[i] > r)
            l = i, r = i + z[i];
    }
    return z;
}
int minRotation(string s) {
    int a=0, N=sz(s); s += s;
    repl(b,0,N) repl(k,0,N) {
        if (a+k == b || s[a+k] < s[b+k]) {b += max
            (0, k-1); break;}
        if (s[a+k] > s[b+k]) {a = b; break; }
    }
    return a;
}
array<vi, 2> manacher(const string& s) {
    int n = sz(s);
    array<vi,2> p = {vi(n+1), vi(n)};
    repl(z,0,2) for (int i=0,l=0,r=0; i < n; i
        ++){
        int t = r-i+!z;

```

```

    if (i<r) p[z][i] = min(t, p[z][l+t]);
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1])
        p[z][i]++, L--, R++;
    if (R>r) l=L, r=R;
}
return p;
}

// segtree
template <class T, T (*op)(T, T), T (*e)()>
struct segtree {
    int n,size;
    vector<T> d;
    segtree(const vector<T>& v){
        n=int(v.size()),size=1;
        while(size<n)size<=1;

        d=vector<T>(2 * size, e());
        for(int i=0;i<n;i++)d[size + i] = v[i];
        for(int i=size-1;i>=1;i--) update(i);
    }
    void update(int k) {
        d[k] = op(d[2 * k], d[2 * k + 1]);
    }
    void set(int p, T x) {
        assert(0 <= p && p < n);
        p += size; d[p] = x;
        while(p > 1)update(p >>=1);
    }
    T prod(int l, int r) const { // [l,r)
        assert(0 <= l && l <= r && r <= n);
        T sml = e(), smr = e();
        for(l += size, r += size ; l<r ; l
            >>=1,r>>=1){
            if (l & 1) sml = op(sml, d[l++]);
            if (r & 1) smr = op(d[--r], smr);
        }
        return op(sml, smr);
    }
};

ll op(ll x, ll y){ return x+y;}
ll e(){return 0ll ;}

template <class S,
    S (*op)(S, S),
    S (*e)(),
    class F,
    S (*Map)(F, S),
    F (*Com)(F, F),
    F (*id)()>
struct lazy_segtree {

    lazy_segtree(const std::vector<S>& v) : _n
        (int(v.size())) {
        size = 1 ;while(size<_n)size*=2 ;
        log = 31 - __builtin_clz(size);
        d = std::vector<S>(2 * size, e());
        lz = std::vector<F>(size, id());
        for (int i = 0; i < _n; i++) d[size +
            i] = v[i];
        for (int i = size - 1; i >= 1; i--)
            update(i);
    }

    void set(int p, S x) {

```

```

        assert(0 <= p && p < _n);
        p += size;
        for (int i = log; i >= 1; i--) push(p
            >> i);
        d[p] = x;
        for (int i = 1; i <= log; i++) update(
            p >> i);
    }

    S get(int p) {
        assert(0 <= p && p < _n);
        p += size;
        for (int i = log; i >= 1; i--) push(p
            >> i);
        return d[p];
    }

    S prod(int l, int r) { // [l,r)
        assert(0 <= l && l <= r && r <= _n);
        if (l == r) return e();

        l += size;
        r += size;

        for (int i = log; i >= 1; i--) {
            if (((l >> i) << i) != 1) push(l
                >> i);
            if (((r >> i) << i) != r) push((r
                - 1) >> i);
        }

        S sml = e(), smr = e();
        while (l < r) {
            if (l & 1) sml = op(sml, d[l++]);
            if (r & 1) smr = op(d[--r], smr);
            l >>= 1;
            r >>= 1;
        }

        return op(sml, smr);
    }

    S all_prod() { return d[1]; }

    void apply(int p, F f) {
        assert(0 <= p && p < _n);
        p += size;
        for (int i = log; i >= 1; i--) push(p
            >> i);
        d[p] = Map(f, d[p]);
        for (int i = 1; i <= log; i++) update(
            p >> i);
    }

    void apply(int l, int r, F f) { // [l,r)
        assert(0 <= l && l <= r && r <= _n);
        if (l == r) return;

        l += size;
        r += size;

        for (int i = log; i >= 1; i--) {
            if (((l >> i) << i) != 1) push(l
                >> i);
            if (((r >> i) << i) != r) push((r
                - 1) >> i);
        }

        {
            int l2 = l, r2 = r;

```

```

            while (l < r) {
                if (l & 1) all_apply(l++, f);
                if (r & 1) all_apply(--r, f);
                l >>= 1;
                r >>= 1;
            }
            l = l2;
            r = r2;
        }

        for (int i = 1; i <= log; i++) {
            if (((l >> i) << i) != 1) update(l
                >> i);
            if (((r >> i) << i) != r) update((
                r - 1) >> i);
        }
    }

    template <bool (*g)(S)> int max_right(int
        l) {
        return max_right(l, [] (S x) { return g
            (x); });
    }

    template <class G> int max_right(int l, G
        g) {
        assert(0 <= l && l <= _n);
        assert(g(e()));
        if (l == _n) return _n;
        l += size;
        for (int i = log; i >= 1; i--) push(l
            >> i);
        S sm = e();
        do {
            while (l % 2 == 0) l >>= 1;
            if (!g(op(sm, d[l]))) {
                while (l < size) {
                    push(l);
                    l = (2 * l);
                    if (g(op(sm, d[l]))) {
                        sm = op(sm, d[l]);
                        l++;
                    }
                }
                return l - size;
            }
            sm = op(sm, d[l]);
            l++;
        } while ((l & -l) != 1);
        return _n;
    }

    template <bool (*g)(S)> int min_left(int r
        ) {
        return min_left(r, [] (S x) { return g(
            x); });
    }

    template <class G> int min_left(int r, G g
        ) {
        assert(0 <= r && r <= _n);
        assert(g(e()));
        if (r == 0) return 0;
        r += size;
        for (int i = log; i >= 1; i--) push((r
            - 1) >> i);
        S sm = e();
        do {
            r--;
            while (r > 1 && (r % 2)) r >>= 1;
            if (!g(op(d[r], sm))) {

```

```

                while (r < size) {
                    push(r);
                    r = (2 * r + 1);
                    if (g(op(d[r], sm))) {
                        sm = op(d[r], sm);
                        r--;
                    }
                }
                return r + 1 - size;
            }
            sm = op(d[r], sm);
        } while ((r & -r) != r);
        return 0;
    }

private:
    int _n, size, log;
    std::vector<S> d;
    std::vector<F> lz;

    void update(int k) { d[k] = op(d[2 * k], d
        [2 * k + 1]); }
    void all_apply(int k, F f) {
        d[k] = Map(f, d[k]);
        if (k < size) lz[k] = Com(f, lz[k]);
    }
    void push(int k) {
        all_apply(2 * k, lz[k]);
        all_apply(2 * k + 1, lz[k]);
        lz[k] = id();
    }
};

// [[ Assignment and Sum ]]
using S = pll ; using F = ll ;
S op( S a, S b) {return {a.ff+b.ff,a.ss+b.ss} ;
}
S e() {return {0,0}; }
S Map(F f, S x) { if(f==-1)return x;
else return {f*x.ss ,x.ss};}
F Com(F f, F g) { return f==-1?g:f ;}
F id() { return -1 ;}

// [[ Addition and Sum ]]
using S = pll; using F = ll ;
S op( S a, S b) { return {a.ff+b.ff,a.ss+b.ss}
; }
S e() { return {0,0}; }
S Map(F f, S x) {return {x.ff + (x.ss*f ),x.ss
};}
F Com(F f, F g) { return f+g ;}
F id() { return 0; }

// psegtree
struct node{
    int64_t sum ;
    node *lc; node *rc;
    node(){ rc=lc=NULL;sum =0 ;
    }
};
node* build(int L,int R){
    node * cur = new node() ;
    if(L == R){
        cur->sum=a[L] ;
        return cur;
    }
    int m=(L+R)/2;
    cur->lc = build(L,m);
    cur->rc= build(m+1,R);

```

```

    cur->sum = cur->lc-> sum + cur->rc->sum ;
    return cur ;
}
ll query (node *cur, int a,int b, int L,int R)
{
    if(a>R || b< L )return 0ll ;
    if(L>= a && R<=b )return cur-> sum ;
    int m=(L+R)/2;
    ll p1 =quary (cur->lc ,a,b ,L,m);
    ll q1 =quary( cur->rc,a,b ,m+1,R);
    return p1+ q1 ;
}
node* update(node *cur ,int i,int val ,int L,
    int R){
    if(L==R ){
        node * ne = new node() ;
        ne ->sum= val ;
        return ne ;
    }
    int m=(L+R)/2;
    if( i<= m ){
        node * ne = new node() ;
        ne-> lc =update(cur->lc, i,val ,L,m) ;
        ne-> rc = cur-> rc ;
        ne-> sum = ne->lc-> sum + ne->rc->sum ;
        return ne ;
    }else{
        node * ne = new node() ;
        ne-> rc =update(cur->rc, i,val ,m+1,R) ;

        ne-> lc = cur-> lc ;
        ne-> sum = ne->lc-> sum + ne->rc->sum ;
        return ne ;
    }
}

// -----HLD-----
// build ord[ Id[i] ] = a[i] for tree
template <bool VALS_EDGES = false >
struct HLD {
    int N, tim = 0;
    vector<vi> adj;
    vi par, siz, rt, pos;

    HLD(vector<vi> adj_)
        : N(sz(adj_)), adj(adj_), par(N, -1), siz(
            N, 1),
            rt(N),pos(N) { dfsSz(0); dfsHld(0); }
    void dfsSz(int v) {
        for (int& u : adj[v]) {
            adj[u].erase(find(all(adj[u]), v));
            par[u] = v;
            dfsSz(u);
            siz[v] += siz[u];
            if (siz[u] > siz[adj[v][0]]) swap(u, adj
                [v][0]);
        }
    }
    void dfsHld(int v) {
        pos[v] = tim++;
        for (int u : adj[v]) {
            rt[u] = (u == adj[v][0] ? rt[v] : u);
            dfsHld(u);
        }
    }
    void process(int u, int v, vii &op) {
        for (; v = par[rt[v]]) {

```

```

            if (pos[u] > pos[v]) swap(u, v);
            if (rt[u] == rt[v]) break;
            op.pb({pos[rt[v]], pos[v] + 1});
        }
        op.pb({pos[u] + VALS_EDGES, pos[v] + 1});
    }
    vii Path(int u, int v) { // return vec of [l
        ,r)
        vii op ; process(u, v,op) ;
        return op ;
    }
    int Id (int v){ // return i such ord[i] =
        a[i] for tree
        return pos[v] ;
    }
    pii querySubtree(int v) { // return [l,r)
        return {pos[v] + VALS_EDGES, pos[v] + siz
            [v]} ;
    }
}
// mo

void MO_gary(){
    /*
        mo tree quary make 2n tour vec
        for path u to v if u == lca(u,v) then quary
            = in[u] to in [v] ;
        else quary out[u] to in[v] then add lca
            node
            all are 0 base
    */
    const int Block = 335 ;
    const int N = 100009;

    struct MO{
        int l,r , id ;
        bool operator<(const MO &e )const {
            return make_pair(l/Block , r)< make_pair(e
                .l/Block ,e.r) ;
        }
    } quary[N];
    /*
        void add (int id){ }
        void rem(int id){ }
        int clc(){ }
    */
    rep(i,q ){ // number of quary
        vi tour , in(n),out(n) ;

        auto dfs=[&](auto dfs,int v,int pa)-> void
            {
                in[v]=sz(tour);
                tour.pb(v) ;
                for(auto u: G[v]){
                    if(u!= pa ){
                        dfs(dfs,u,v) ;
                    }
                }
                out[v]=sz(tour);
                tour.pb(v);
                ; dfs(dfs,0,-1);
            }

        rep(i,q ){ // number of quary
            int u , v ;
            cin>>u>>v ;
            --u,--v;
            if(in[u]> in[v] )swap(u,v) ;

```

```

            int x= A.lca(u,v) ;
            if( x == u){
                quary[i]={in[u],in[v],i};
            }else{
                quary[i]={out[u],in[v],i};
            }
        }
        sort (quary,quary+q);

        int cl = 0 ,cr= -1 ;
        vi ans (q) ;
        for (int i=0;i<q;i++) {
            auto &[l,r ,k, lca, id ] = quary[i];
            while (cl > l)add( --cl);
            while (cr < r)add(++cr );
            while (cl < l)rem(cl++ );
            while (cr > r)rem (cr-- );
            ans[id]= clc () ;
        }
    }

    void MO_gary(){
        // mo quary 0 base
        const int Block = 335 ,N = 100009;

        struct MO{
            int l,r , id ;
            bool operator<(const MO &e )const {
                return make_pair(l/Block , r)< make_pair
                    (e.l/Block ,e.r) ;
            }
        } quary[N];
        /*
            void add (int id){ }
            void rem(int id){ }
            int clc(){ }
        */
        rep(i,q ){ // number of quary

            int l,r ;
            cin>>l>>r;
            --l,--r;
            quary[i]={l,r,i};
        }
        sort (quary,quary+q);
        int cl = 0 ,cr= -1 ;
        vi ans (q) ;
        for (int i=0;i<q;i++) {
            auto &[l,r ,k, lca, id ] = quary[i];
            while (cl > l)add( --cl);
            while (cr < r)add(++cr );
            while (cl < l)rem(cl++ );
            while (cr > r)rem (cr-- );
            ans[id]= clc () ;
        }
    }
    // segtree sort

    template<class S> struct segtree {
        int n; V<S> t;
        void init(int _) { n = _; t.assign(n+n-1, S
            ()); }
        void init(const V<S>& v) {
            n = sz(v); t.assign(n + n - 1, S());
            build(0,0,n-1,v);
        } template <typename... T>
        void upd(int l, int r, const T&... v) {

```

```

        assert(0 <= l && l <= r && r < n);
        upd(0, 0, n-1, l, r, v...);
    }
    S get(int l, int r) {
        assert(0 <= l && l <= r && r < n);
        return get(0, 0, n-1, l, r);
    }
private:
    inline void push(int u, int b, int e) {
        if (t[u].lazy == 0) return;
        int mid = (b+e)>>1, rc = u+((mid-b+1)<<1);
        t[u+1].upd(b, mid, t[u].lazy);
        t[rc].upd(mid+1, e, t[u].lazy);
        t[u].lazy = 0;
    }
    void build(int u,int b,int e,const V<S>&v) {
        if (b == e) return void(t[u] = v[b]);
        int mid = (b+e)>>1, rc = u+((mid-b+1)<<1);
        build(u+1, b,mid,v); build(rc, mid+1,e,v);
        t[u] = t[u+1] + t[rc];
    } template<typename... T>
    void upd(int u, int b, int e, int l, int r,
        const T&... v) {
        if (l <= b && e <= r) return t[u].upd(b, e
            , v...);
        push(u, b, e);
        int mid = (b+e)>>1, rc = u+((mid-b+1)<<1);
        if (l<=mid) upd(u+1, b, mid, l, r, v...);
        if (mid<r) upd(rc, mid+1, e, l, r, v...);
        t[u] = t[u+1] + t[rc];
    }
    S get(int u, int b, int e, int l, int r) {
        if (l <= b && e <= r) return t[u];
        push(u, b, e);
        S res; int mid = (b+e)>>1, rc = u+((mid-b
            +1)<<1);
        if (r<=mid) res = get(u+1, b, mid, l, r);
        else if (mid<l) res = get(rc,mid+1,e,l,r);
        else res = get(u+1, b, mid, l, r) + get(rc
            , mid+1, e, l, r);
        t[u] = t[u+1] + t[rc]; return res;
    }
}; // Hash upto here = 773c09
/* (1) Declaration:
    Create a node class. Now, segtree<node> T;
    T.init(10) creates everything as node()
    Consider using V<node> leaves to build
    (2) upd(l, r, ...v): update range [l, r]
    order in ...v must be same as node.upd() fn */
struct node {
    ll sum = 0, lazy = 0;
    node () {} // write full constructor
    node operator+(const node &obj) {
        return {sum + obj.sum, 0};
    }
    void upd(int b, int e, ll x) {
        sum += (e - b + 1) * x, lazy += x;
    }
};

```